MIT Art Design and Technology University

MIT School of Computing, Pune

MIT-ADT UNIVERSITY
PUNE, INDIA

23CSE3042 – Deep Learning

Class - T.Y. (SEM-II), AIA

Unit - V Advances in Deep Learning and Applications

AY 2025-2026 SEM-II

Autoencoders

Unit V

Intuition

- Consider a dataset of people faces, where each input is an image of a person. The representation of an image in its raw format is too complex to be used by machines.
- Instead, why not make the neural network automatically calculate important features for each face, something maybe like: eye type, nose type, distance between eyes, nose position, etc.
- Well, this sounds interesting… Using these features, we could easily **compare two faces**, **find similar faces**, **generate new faces**, and many other interesting applications.

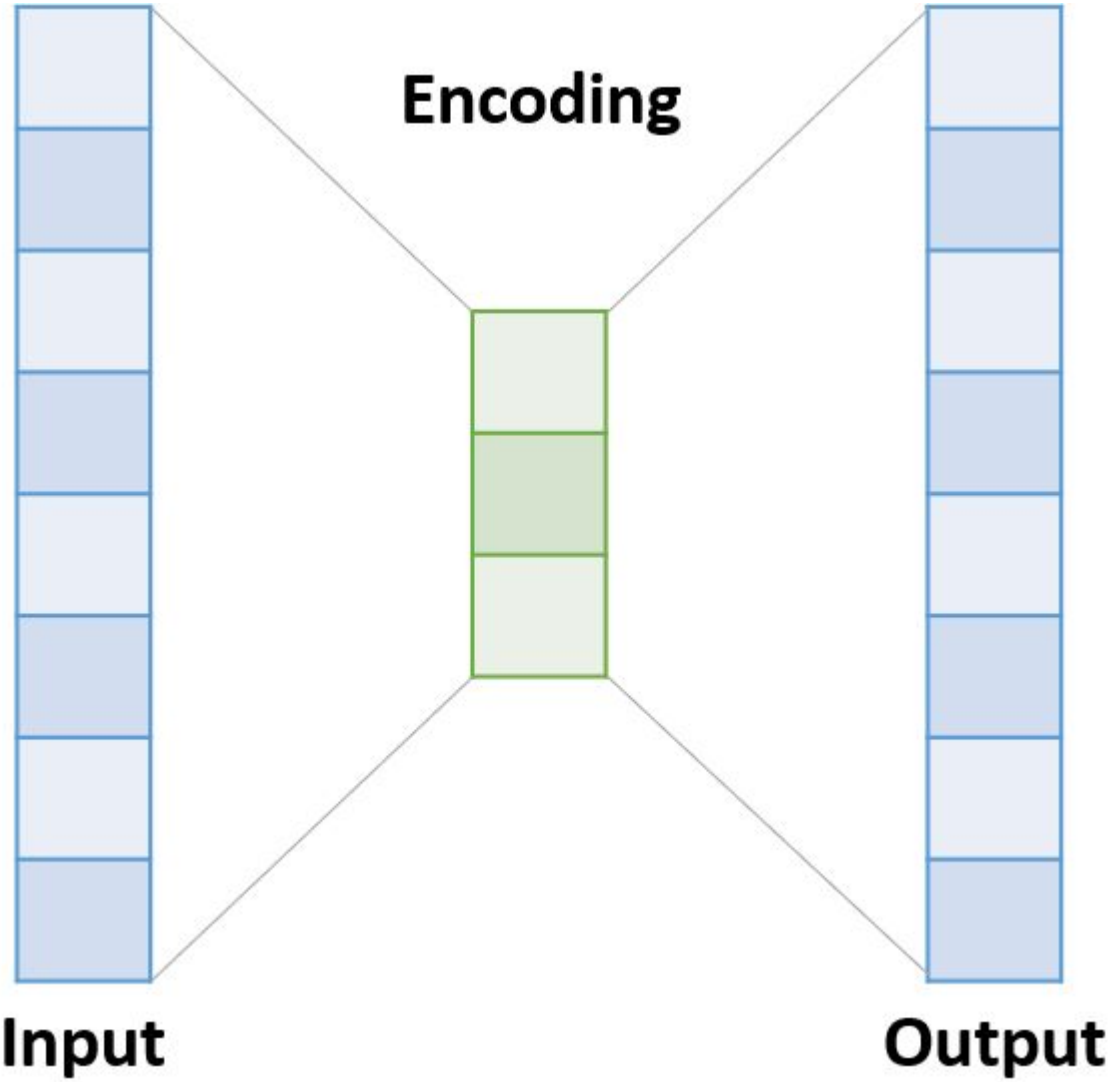


Autoencoders

- An **AutoEncoder** is a neural network
- both its input and output are the same.
- So, it is a network that tries to learn itself!
- Example
  - Suppose we have the following neural network:
  - An input layer with 100 neurons
  - A Hidden layer with 3 neurons
  - An output layer with 100 neurons (same as the input layer)

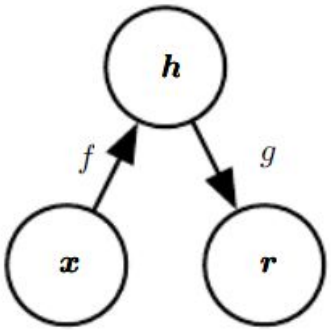


Autoencoders



Autoencoder

- An **autoencoder** is a neural network that is trained to attempt to copy its input to its output
- Internally, it has a hidden layer h that describes a **code** used to represent the input.
- The network may be viewed as consisting of two parts:
  - encoder function h = f (x)
  - decoder function r = g(h).





Autoencoder

- If an autoencoder succeeds in simply learning to set g(f (x)) = x everywhere, then it is not especially useful.
- Instead, autoencoders are designed to be unable to learn to copy perfectly.
- Usually they are restricted in ways that allow them to copy only approximately, and to copy only input that resembles the training data.
- Because the model is forced to prioritize which aspects of the input should be copied, it often learns useful properties of the data.



Autoencoder

- Traditionally used for
  - dimensionality reduction
  - feature learning
- Now used for generative modeling
- AE are special case of FFN
- may be trained with all of the same techniques, typically minibatch gradient descent following gradients computed by back-propagation



Autoencoder

- The simplest architecture for constructing an autoencoder is to constrain the number of nodes present in the hidden layer(s) of the network, limiting the amount of information that can flow through the network.
- By penalizing the network according to the reconstruction error, our model can learn the most important attributes of the input data and how to best reconstruct the original input from an "encoded" state.
- Ideally, this encoding will **learn and describe latent attributes of the input data**.



Undercomplete Autoencoders

- One way to obtain useful features from the autoencoder is to constrain h to have smaller dimension than x
- An autoencoder whose code dimension is less than the input dimension is called **undercomplete**
- Learning an undercomplete representation forces the autoencoder to capture the most salient features of the training data.
- The learning process is described simply as minimizing a loss function

$$L(x, g(f(x)))$$

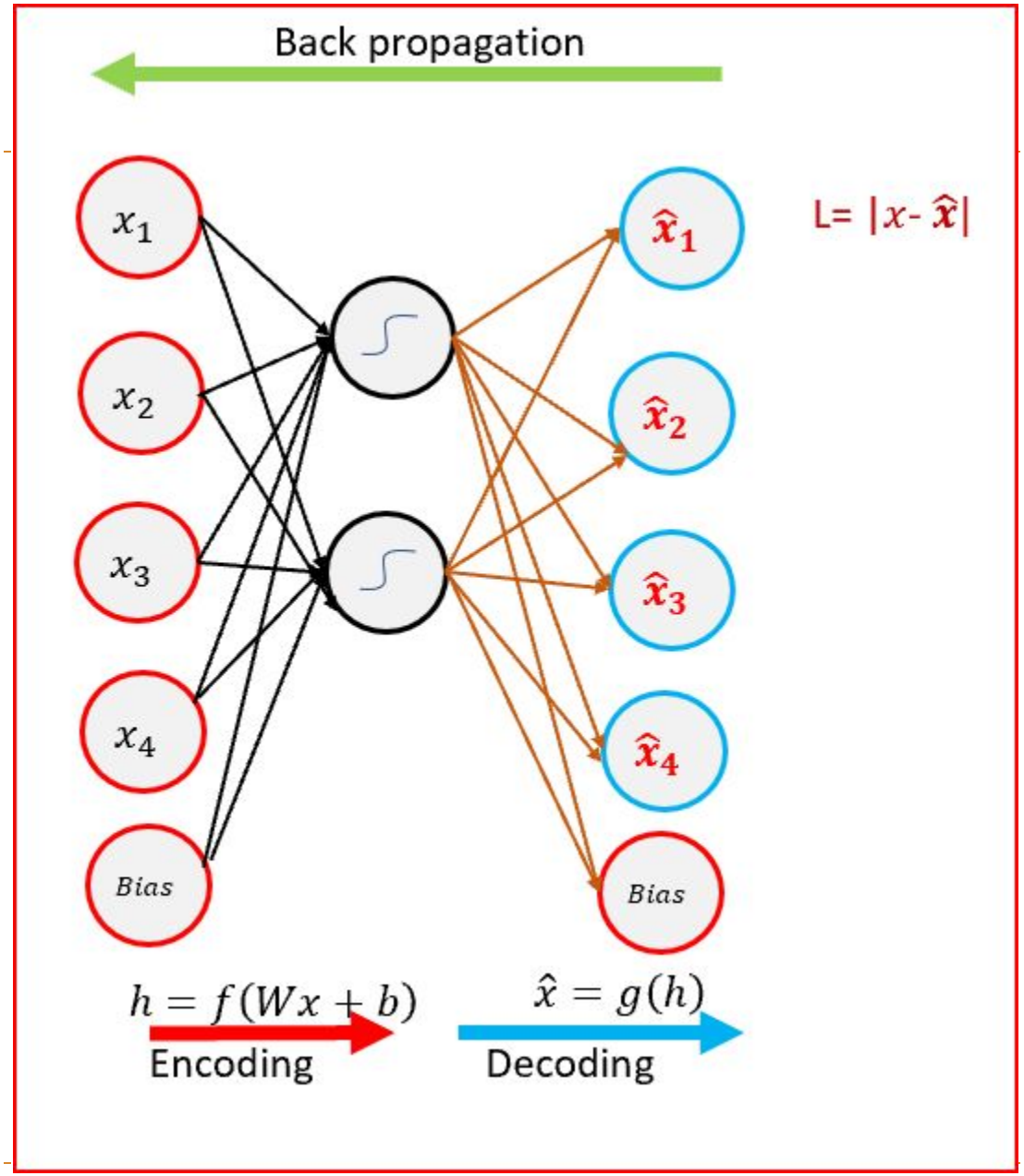
- L is a loss function penalizing g(f (x)) for being dissimilar from x



Undercomplete Autoencoders

- When the decoder is linear and L is the mean squared error
- An undercomplete autoencoder has no explicit regularization term - we simply train our model according to the reconstruction loss.
- Thus, our only way to ensure that the model isn't memorizing the input data is the ensure that we've sufficiently restricted the number of nodes in the hidden layer(s)




Back propagation
x_1
x_2
x_3
x_4
Bias
$\hat{x}_1$
$\hat{x}_2$
$\hat{x}_3$
$\hat{x}_4$
Bias
L= $|x - \hat{x}|$
$h = f(Wx + b)$
Encoding
$\hat{x} = g(h)$
Decoding

Regularized Autoencoder

- Undercomplete autoencoders fail to learn anything useful if the encoder and decoder are given too much capacity.
- A similar problem occurs if the hidden code is allowed to have dimension equal to the input
- The **overcomplete** case in which the hidden code has dimension greater than the input.
- In these cases, even a linear encoder and linear decoder can learn to copy the input to the output without learning anything useful about the data distribution.



Regularized Autoencoder

- A regularized autoencoder can be nonlinear and overcomplete but still learn something useful about the data distribution even if the model capacity is great enough to learn a trivial identity function
- Types
  - Sparse Autoencoders
  - Denoising Autoencoders

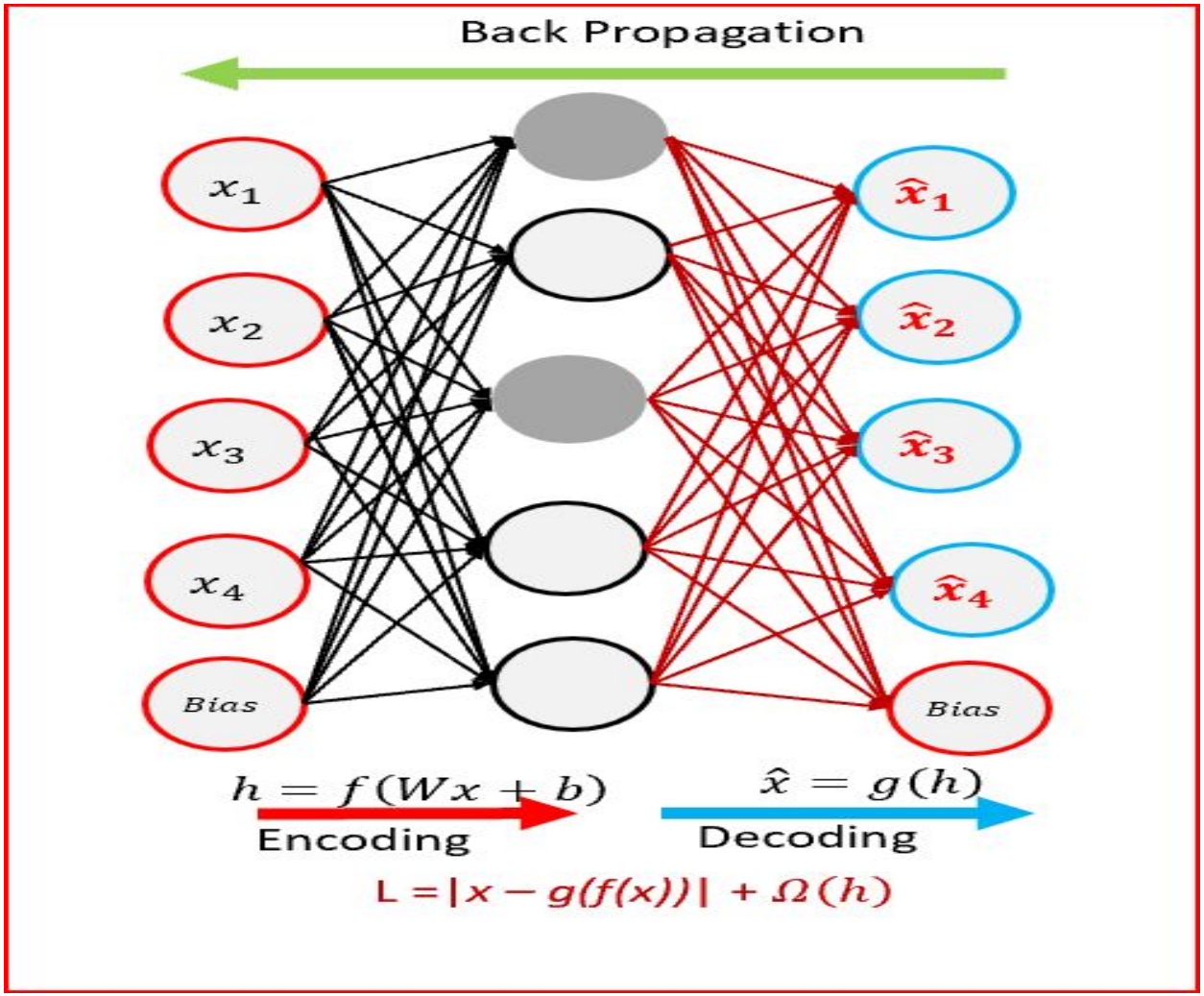


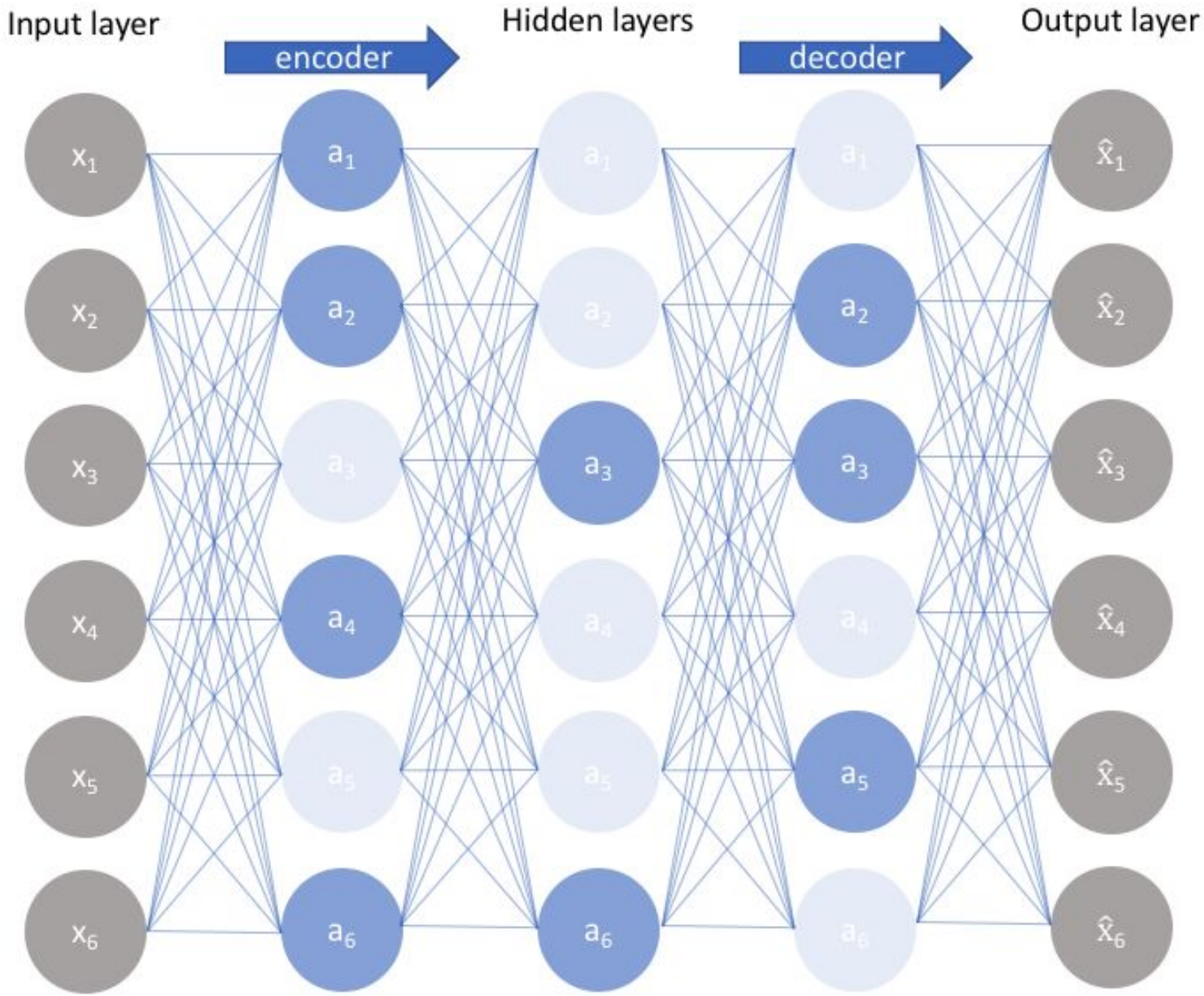
Sparse Autoencoders- Intuition

- The intuition behind this method is that, for example, if a man claims to be an expert in mathematics, computer science, psychology, and classical music, he might be just learning some quite **shallow knowledge** in these subjects.
- However, if he only claims to be devoted to mathematics, we would like to anticipate some **useful insights** from him.
- And it's the same for autoencoders we're training — fewer nodes activating while still keeping its performance would guarantee that the autoencoder is actually learning **latent representations** instead of redundant information in our input data.



Sparse Autoencoders




Input layer
encoder
Hidden layers
decoder
Output layer
x_1
x_2
x_3
x_4
x_5
x_6
a_1
a_2
a_3
a_4
a_5
a_6
a_1
a_2
a_3
a_4
a_5
a_6
a_1
a_2
a_3
a_4
a_5
a_6
$\hat{x}_1$
$\hat{x}_2$
$\hat{x}_3$
$\hat{x}_4$
$\hat{x}_5$
$\hat{x}_6$

Sparse Autoencoders

- A sparse autoencoder is simply an autoencoder whose training criterion involves a sparsity penalty Ω(h) on the code layer h, in addition to the reconstruction error

$$L(x, g(f(x))) + \Omega(h)$$

- Sparse autoencoders are typically used to learn features for another task such as classification
- Regularizers: L1 and KL Divergence



L1 Regularaliza tion

- We can add a term to our loss function that penalizes the absolute value of the vector of activations aa in layer h for observation i, scaled by a tuning parameter λ.

$$\mathcal{L}(x, \hat{x}) + \lambda \sum_i \left| a_i^{(h)} \right|$$

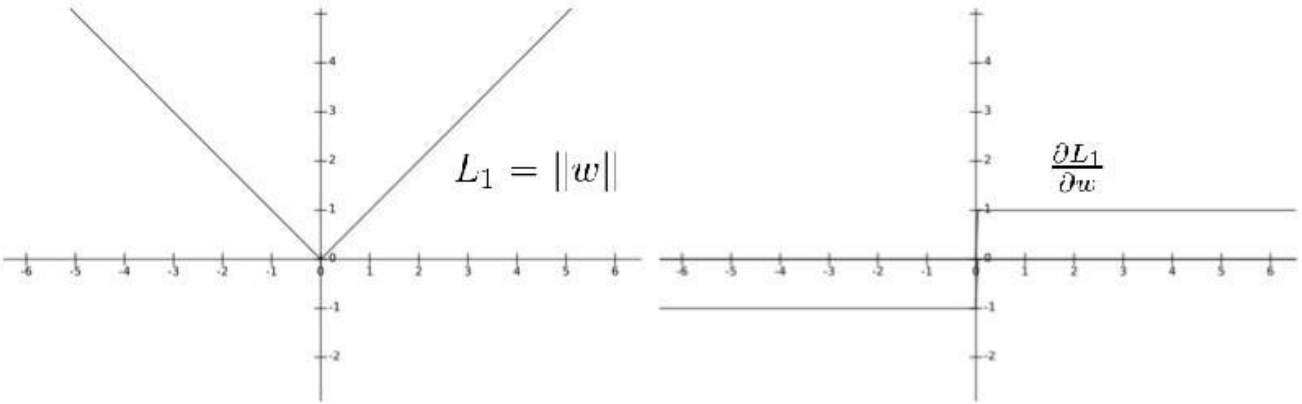
- the key difference between them is that L1 regularization tends to shrink the penalty coefficient to zero while L2 regularization would move coefficients towards zero but they will never reach.
- Thus L1 regularization is often used as a method of feature extraction

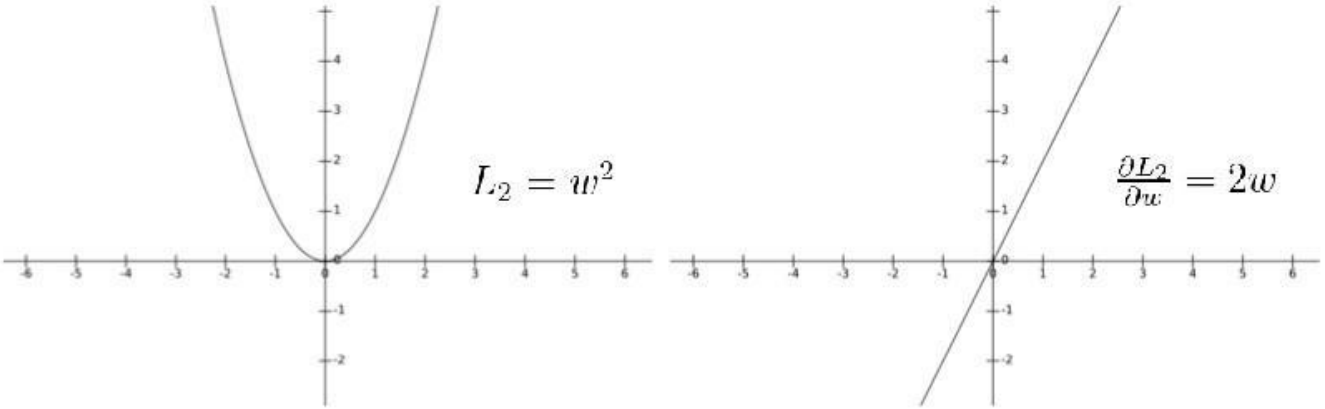


- Consider that we have two loss functions L1 and L2 which represent L1 regularization and L2 regularization respectively.

$$L_1 = \|w\| \, , L_2 = w^2$$

- If we plot these two loss functions and their derivatives, it looks like this:





- for L1 regularization, the gradient is either 1 or -1 except when w=0, which means that L1 regularization will always move w towards zero with **same step size** (1 or -1) regardless of the value of w. And when w=0, the gradient becomes zero and no update will be made anymore.
- However, for L2 regularization things are different. L2 regularization will also move w towards zero but the step size becomes smaller and smaller which means that w will **never reach zero**



KL Divergence

- we will think of a neuron as being "active" (or as "firing") if its output value is close to 1, or as being "inactive" if its output value is close to 0.
- $a^{(2)}_j$ denotes the activation of hidden unit j in the autoencoder.
- $a^{(2)}_j(x)$ to denote the activation of this hidden unit when the network is given a specific input x.

$$\hat{\rho}_j = \frac{1}{m} \sum_{i=1}^{m} \left[a_j^{(2)}(x^{(i)}) \right]$$

- be the average activation of hidden unit j (averaged over the training set).
- We would like to (approximately) enforce the constraint

$$\hat{\rho}_j = \rho,$$



KL Divergence

- where ρ is a sparsity parameter, typically a small value close to zero (say ρ = 0.05).
- In other words, we would like the average activation of each hidden neuron j to be close to 0.05.
- To satisfy this constraint, the hidden unit's activations must mostly be near 0
- To achieve this, we will add an extra penalty term to our optimization objective that penalizes ˆρj deviating significantly from ρ.

$$\sum_{j=1}^{s_2} \rho \log \frac{\rho}{\hat{\rho}_j} + (1-\rho) \log \frac{1-\rho}{1-\hat{\rho}_j}$$

- s_2 is the number of neurons in the hidden layer, and the index j is summing over the hidden units in our network



KL Divergence

- KL divergence is written as

$$\sum_{j=1}^{s_2} \mathrm{KL}(\rho||\hat{\rho}_j)$$

- where $\mathrm{KL}(\rho||\hat{\rho}_j) = \rho \log \frac{\rho}{\hat{\rho}_j} + (1-\rho) \log \frac{1-\rho}{1-\hat{\rho}_j}$
- KL-divergence is a standard function for measuring how different two different distributions are.
- This penalty function has the property that,

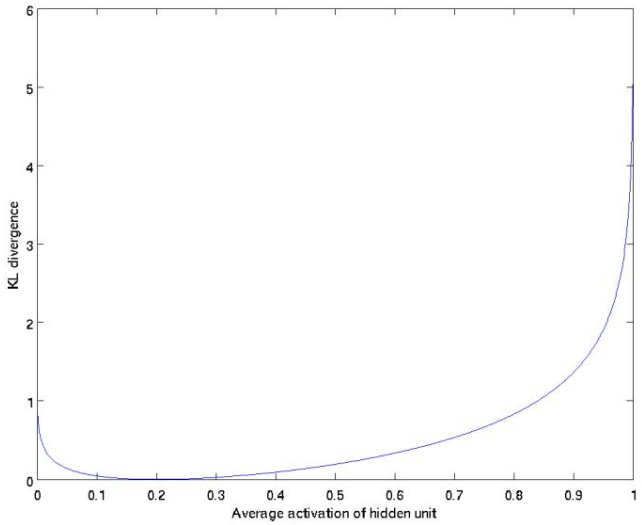
$$\mathrm{KL}(\rho||\hat{\rho}_j) = 0 \text{ if } \hat{\rho}_j = \rho$$

- Otherwise it increases monotonically as ˆρj diverges from ρ.



KL Divergence

- For example, in the figure below, we have set ρ = 0.2, and plotted KL(ρ| |ρˆj) for a range of values of ˆρj :



- We see that the KL-divergence reaches its minimum of 0 at ˆρj = ρ, and blows up (it actually approaches ∞) as ˆρj approaches 0 or 1. Thus, minimizing this penalty term has the effect of causing ˆρj to be close to ρ.

$$J_{\text{sparse}}(W,b) = J(W,b) + \beta \sum_{j=1}^{s_2} \text{KL}(\rho||\hat{\rho}_j)$$

Denoising Autoencoders

- Rather than adding a penalty Ω to the cost function, we can obtain an autoencoder that learns something useful by changing the reconstruction error term of the cost function.
- autoencoders minimize some function

$$L(x, g(f(x)))$$

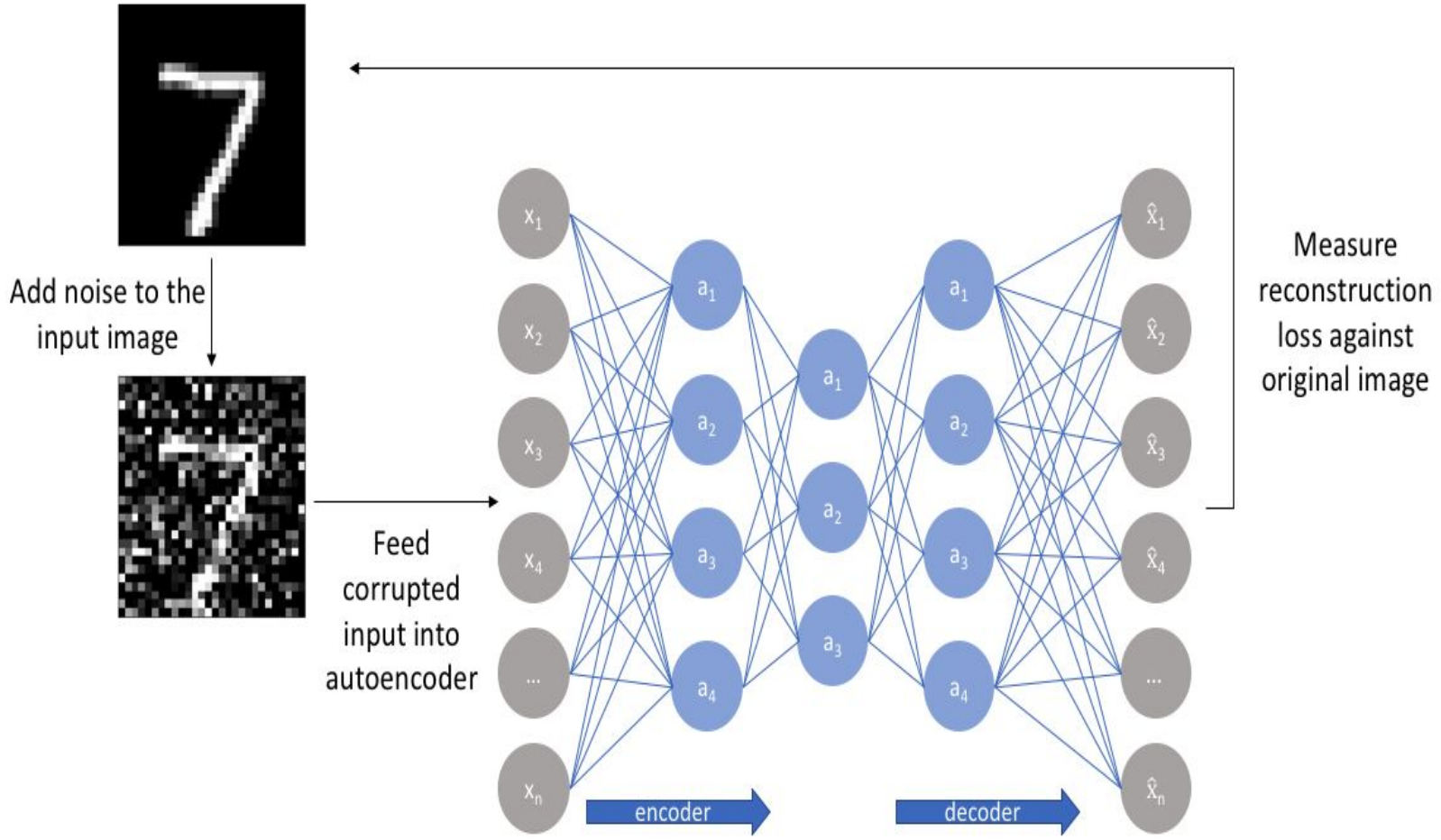
- A **denoising autoencoder** or DAE instead minimizes

$$L(x, g(f(\tilde{x})))$$

- where $\tilde{x}$ is a copy of x that has been corrupted by some form of noise.
- Denoising autoencoders must therefore undo this corruption rather than simply copying their input.



Denoising Autoencoders



Representational Power, Layer Size and Depth

- Autoencoders are often trained with only a single layer encoder and a single layer decoder.
- Deep encoder and decoders
- Advantages to depth in a feedforward network?
- Because autoencoders are feedforward networks, these advantages also applicable to autoencoders.



Representational Power, Layer Size and Depth

- One major advantage of non-trivial depth is that the universal approximator theorem guarantees that a feedforward neural network with at least one hidden layer can represent an approximation of any function to an arbitrary degree of accuracy, provided that it has enough hidden units

- Depth can exponentially reduce the computational cost of representing some functions.

- deep autoencoders yield much better compression than corresponding shallow or linear autoencoders



Denoising Auto-Encoder?

Input and output data of the auto-encoder is basically **identical**.

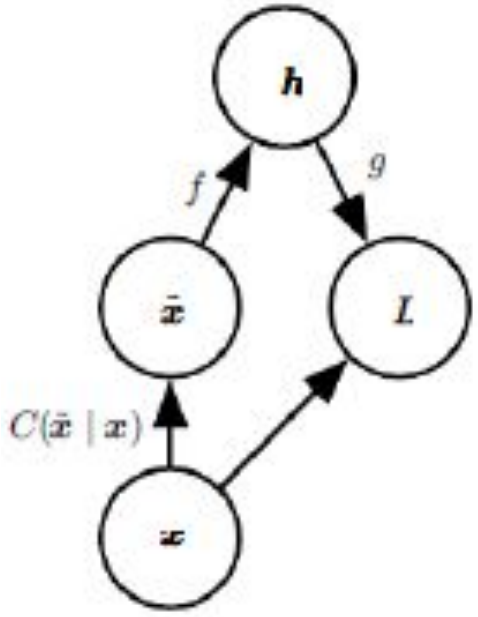
Then, how can we make this network to have **denoising power**?





Denoising Autoencoder

- The **denoising autoencoder** (DAE) is an autoencoder that receives a corrupted data point as input
- and is trained to predict the original, uncorrupted data point as its output

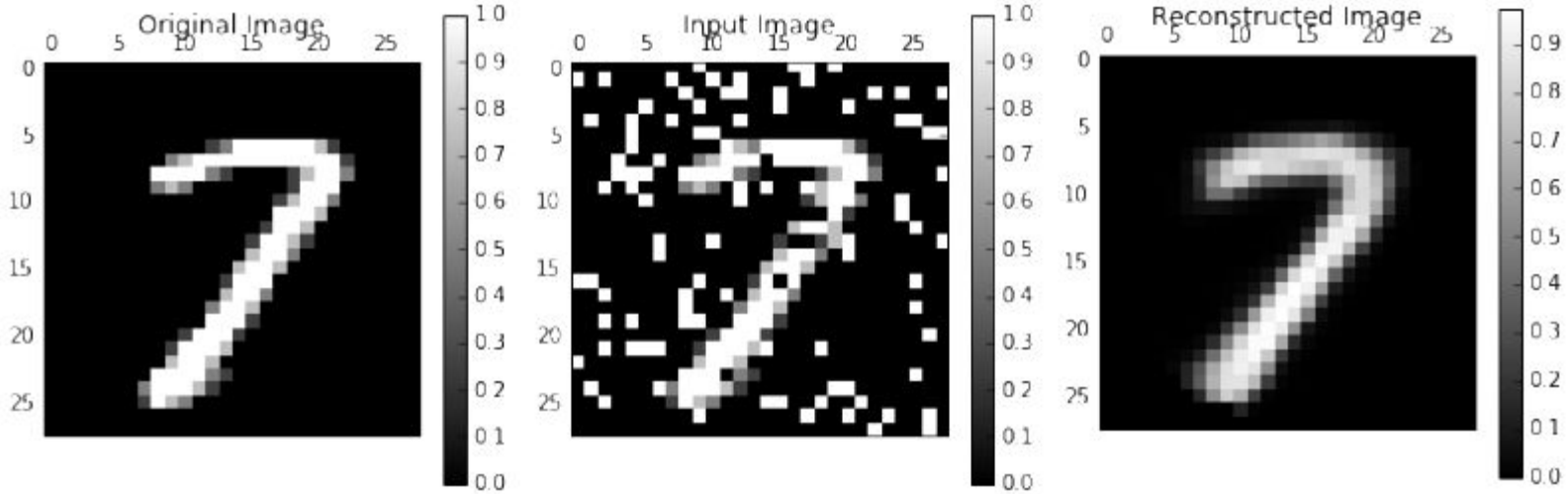


corruption process C(˜x | x) which represents a conditional distribution over corrupted samples ˜x, given a data sample x.

Denoising Autoencoder

- The autoencoder then learns a **reconstruction distribution** $p_{reconstruct}(x \mid \tilde{x})$ estimated from training pairs $(x, \tilde{x})$, as follows:
  - Sample a training example x from the training data.
  - Sample a corrupted version $\tilde{x}$ from $C(\tilde{x} \mid x = x)$.
  - Use $(x,\tilde{x})$ as a training example for estimating the autoencoder reconstruction distribution $p_{reconstruct}(x \mid \tilde{x}) = p_{decoder}(x \mid h)$ with h the output of encoder $f(\tilde{x})$ and $p_{decoder}$ typically defined by a decoder g(h).
- perform gradient-based approximate minimization (such as minibatch gradient descent) on the negative log-likelihood $-\log p_{decoder}(x \mid h)$.




Original Image
Input Image
Reconstructed Image

Contractive Autoencoders

- The contractive autoencoder introduces an explicit regularizer on the code h = f(x), encouraging the derivatives of f to be as small as possible

$$\Omega(\boldsymbol{h}) = \lambda \left\| \frac{\partial f(\boldsymbol{x})}{\partial \boldsymbol{x}} \right\|_F^2$$

- The penalty Ω(h) is the squared Frobenius norm (sum of squared elements) of the Jacobian matrix of partial derivatives associated with the encoder function
- What is connection between contractive and denoising autoencoders?



Contractive Autoencoders

- In the limit of small Gaussian input noise, the denoising reconstruction error is equivalent to a contractive penalty on the reconstruction function that maps x to r = g(f(x)).
- Denoising autoencoders make the reconstruction function resist small but finite-sized perturbations of the input
- while contractive autoencoders make the feature extraction function resist infinitesimal perturbations of the input



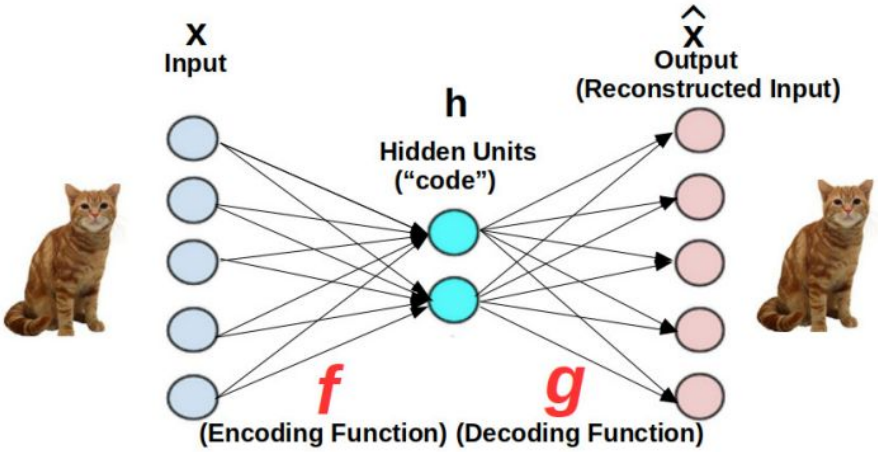
- Applications of Autoencoders
- Deep Generative Models
- Deep Learning Applications



Autoencoder

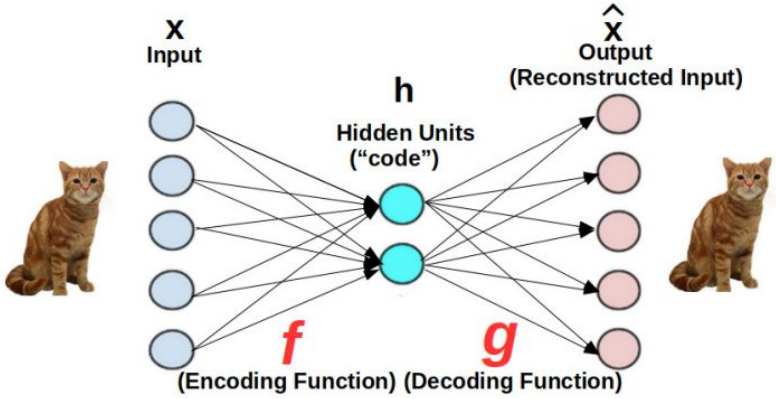
Autoencoder

- Similar to the standard feedforward neural network with a key difference:
- Unsupervised. No “label” at the output layer; Output layer simply tries to “recreate” the input

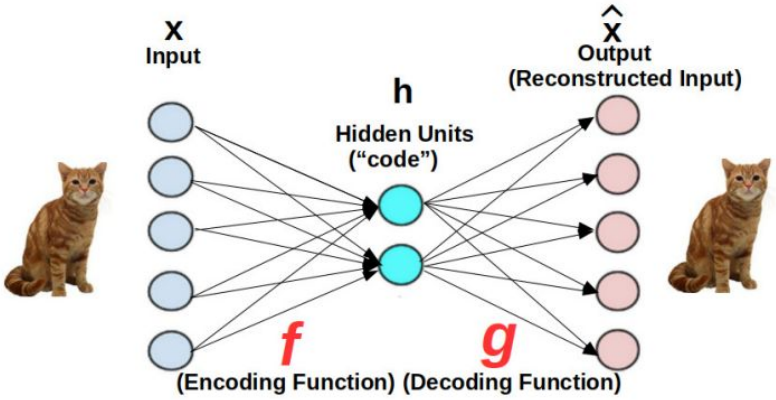


Autoencoder

- Defined by two (possibly nonlinear) mapping functions: Encoding function f , Decoding function g
- h = f (x) denotes an encoding (possibly nonlinear) for the input x
- x^ = g(h) = g(f (x)) denotes the reconstruction (or the “decoding”) for the input x
- For an Autoencoder, f and g are learned with a goal to minimize the difference between x^ and x



- The learned code h = f (x) can be used as a new feature representation of the input x.
- Therefore autoencoders can also be used for “feature learning”.

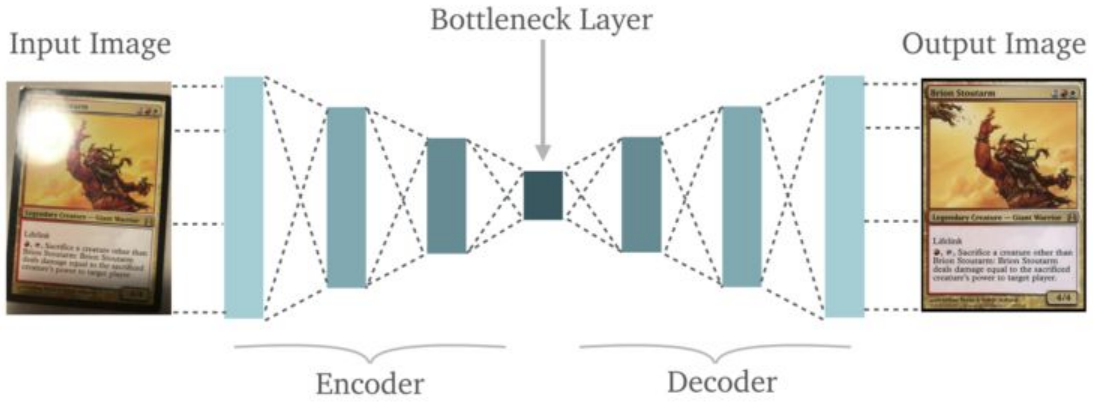


Applications of autoencoders

- Image Coloring
- Autoencoders are used for converting any black and white picture into a colored image. Depending on what is in the picture, it is possible to tell what the color should be



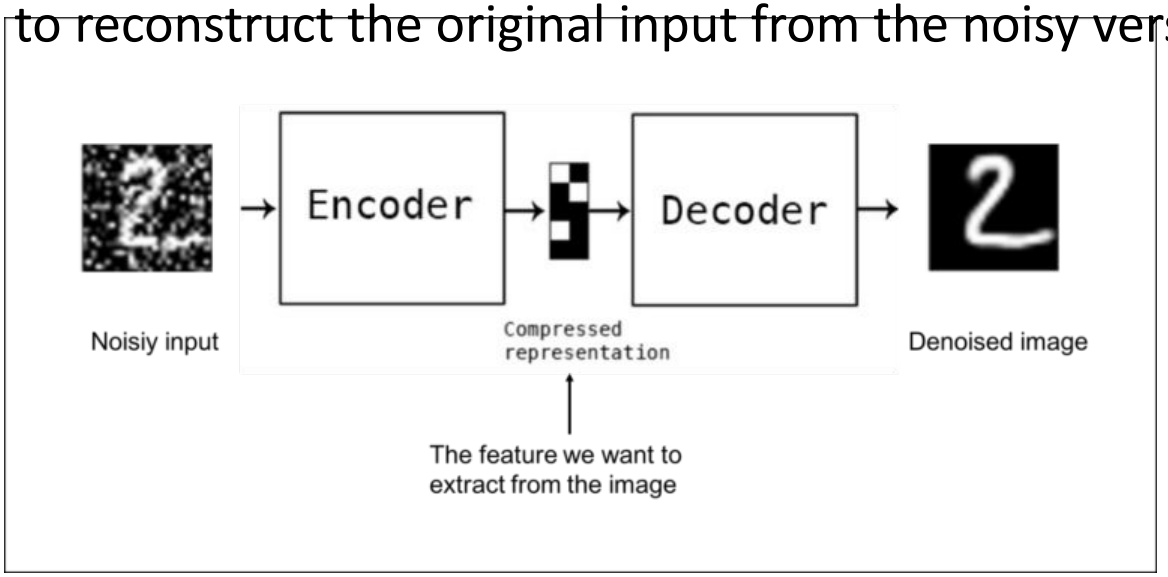
- Feature variation
- It extracts only the required features of an image and generates the output by removing any noise or unnecessary interruption.



- Dimensionality Reduction
- The reconstructed image is the same as our input but with reduced dimensions. It helps in providing the similar image with a reduced pixel value.



- Denoising Image
- The input seen by the autoencoder is not the raw input but a stochastically corrupted version. A denoising autoencoder is thus trained to reconstruct the original input from the noisy version.



- Watermark Removal
- It is also used for removing watermarks from images or to remove any object while filming a video or a movie.



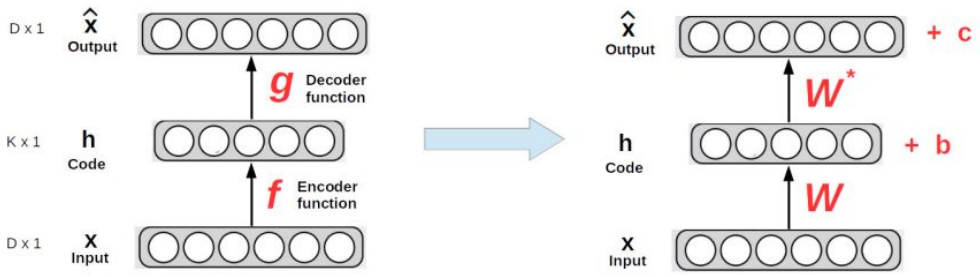
A Simple Autoencoder

- Let's assume a D × 1 input $x \in R^D$, and a single hidden layer with K × 1 code $h \in R^K$
- We can then define a simple linear autoencoder as

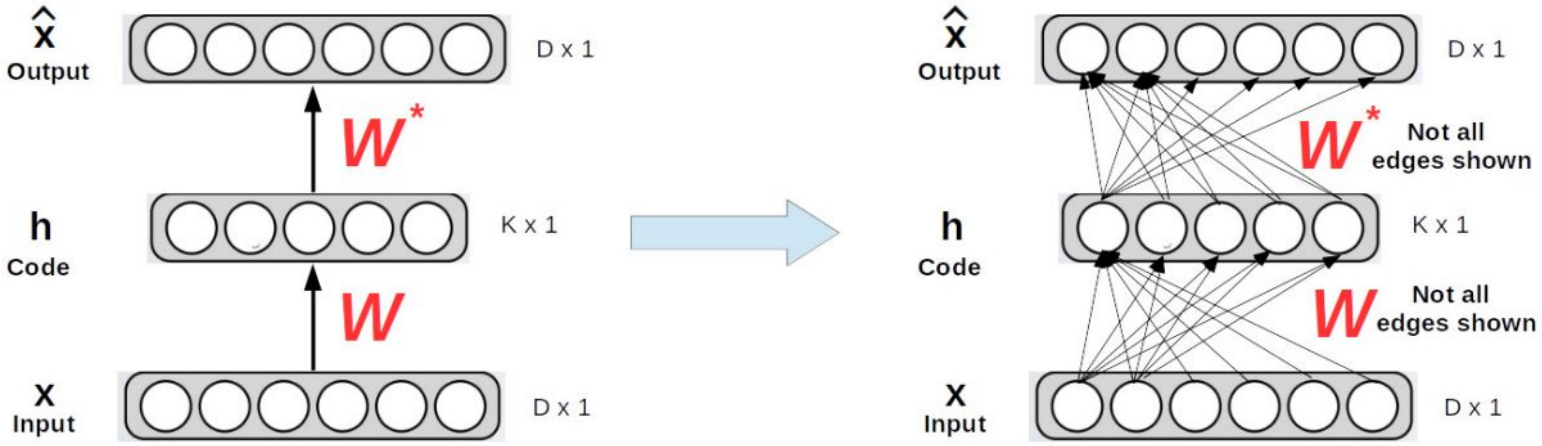
$$h = f(x) = Wx + b$$

$$\hat{x} = g(h) = W^*h + c$$

- where f is defined by $W \in R^{K \times D}$ and $b \in R^{K \times 1}$, g is defined by $W^* \in R^{D \times K}$ and $c \in R^{D \times 1}$
- Note: If we learn f , near
  autoencoder with W onent
  Analysis (PCA)



Autoencoder: Zooming in..



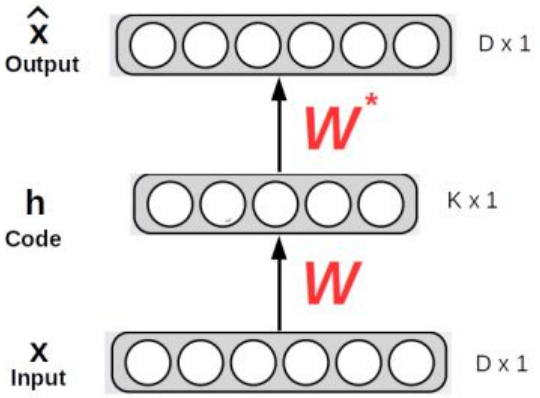
W: K × D matrix of weights of edges between input and hidden layer.
W_{kd} is the weight of edge connecting input layer node d to hidden layer node k.

W^* : D × K matrix of weights of edges between hidden and output layer.
W^*_{dk} is the weight of edge connecting hidden layer node k to output layer node d.

If $W^* = W^T$, the autoencoder architecture is said to have "tied weights"

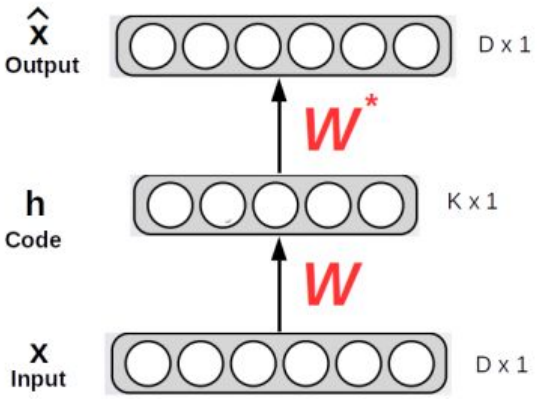
Nonlinear Autoencoders

- The hidden nodes can also be nonlinear transforms of the inputs, e.g.,
- Can define h as a linear transform of x followed by a nonlinearity (e.g., sigmoid, ReLU)
- h = sigmoid(Wx + b)
- where the nonline p(−z)) squashes the real-valued z to lie



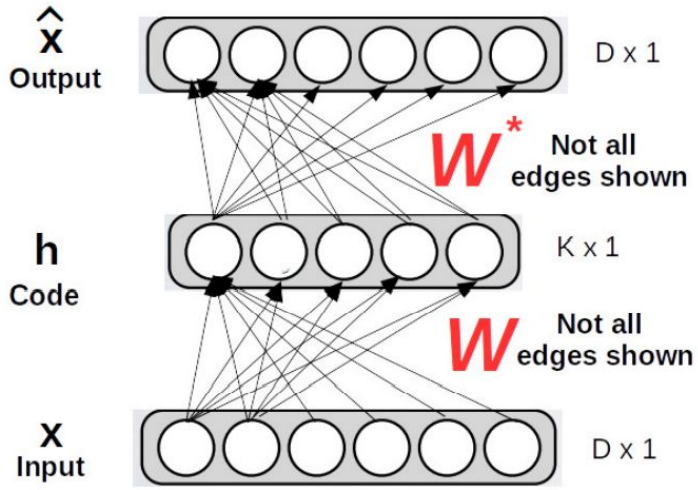
- Note: If inputs $x \in \{0, 1\}^D$ are binary, it may be more appropriate to also define $x^$ as

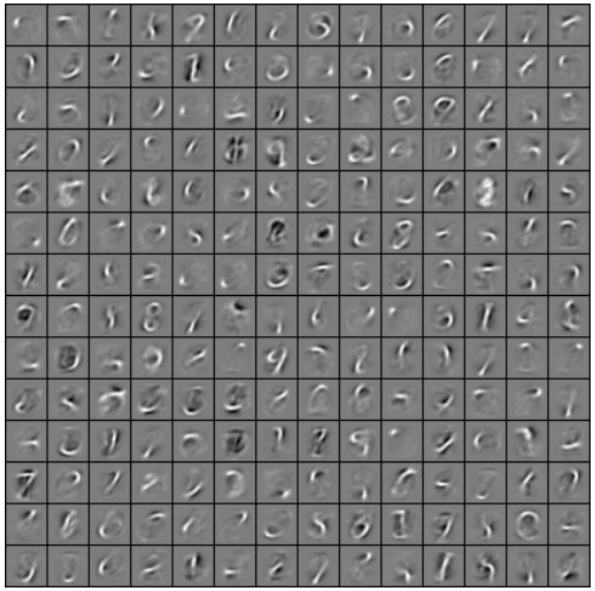
$$x^ = \text{sigmoid}(W^*h + c)$$



What's Learned by an Autoencoder?

- The K × D matrix W learned on digits data. Each tiny block visualizes a row of W.
- Thus W captures the possible "patterns" in the training data.





Training the Autoencoder

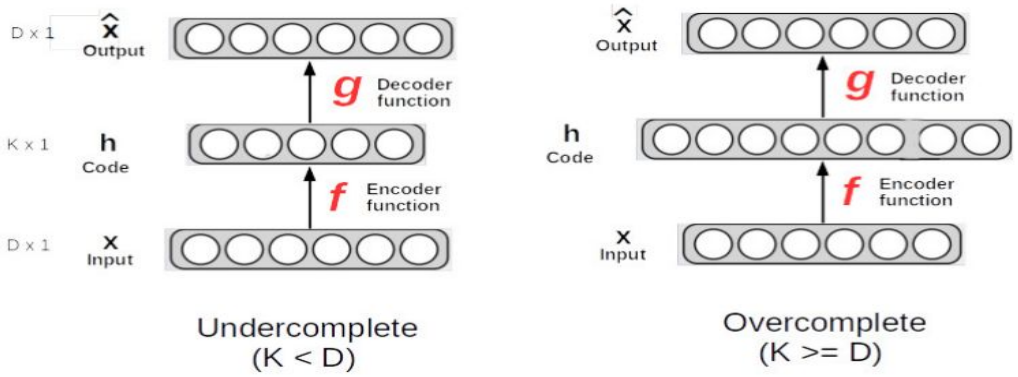
- To train the autoencoder, we need to define a loss function `(x^, x)
- The loss function (a function of parameters W, b, W*, c) can be defined using various ways
- In general, it is defined in terms of the difference between x^ and x (reconstruction error)
- For a single input x = [x1 ... xD] and its reconstruction x^ = [^x1 ... x^D]

$$\ell(\hat{\boldsymbol{x}}, \boldsymbol{x}) = \sum_{d=1}^{D} (\hat{x}_d - x_d)^2 \quad \text{(squared loss; used if input are real-valued)}$$

$$\ell(\hat{\boldsymbol{x}}, \boldsymbol{x}) = -\sum_{d=1}^{D} [x_d \log(\hat{x}_d) + (1 - x_d) \log(1 - \hat{x}_d)] \quad \text{(cross-entropy loss; used if input are binary)}$$

- We find (W, b, W*, c) by minimizing the reconstruction error (summed over all training data)
- This can be done using backpropagation

Undercomplete, Overcomplete, and Need for Regularization



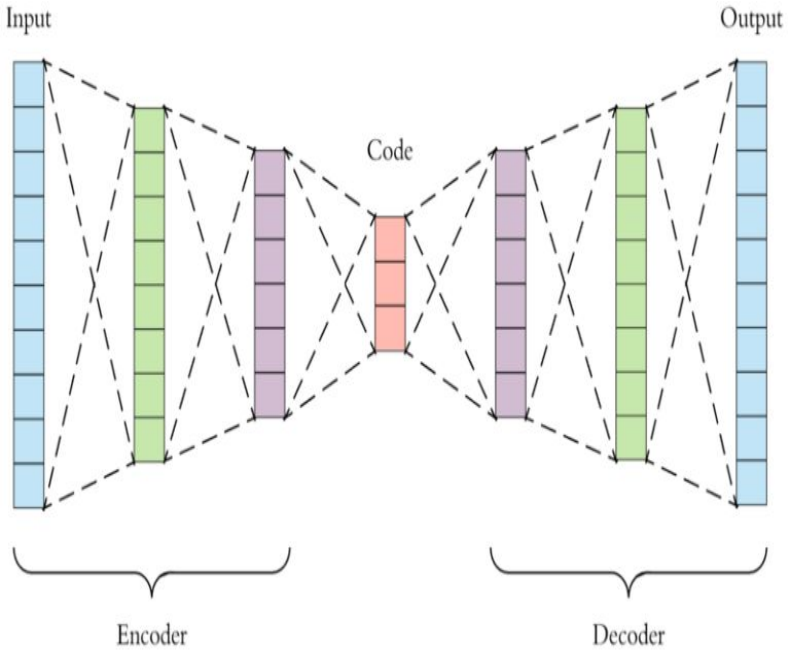
- In both cases, it is important to control the capacity of encoder and decoder
- Undercomplete: Imagine K = 1 and very powerful f and g. Can achieve very small reconstruction error but the learned code will not capture any interesting properties in the data
- Overcomplete: Imagine K ≥ D and trivial (identity) functions f and g. Can achieve even zero reconstruction error but the learned code will not capture any interesting properties in the data
- It is therefore important to regularize the functions as well as the learned code, and not just focus on minimizing the reconstruction error

Properties of Autoencoders:

- Data-specific: Autoencoders are only able to compress data similar to what they have been trained on. It is easy to train specialized instances of the algorithm that will perform well on a specific type of input.
- Lossy: The decompressed outputs will be degraded compared to the original inputs.

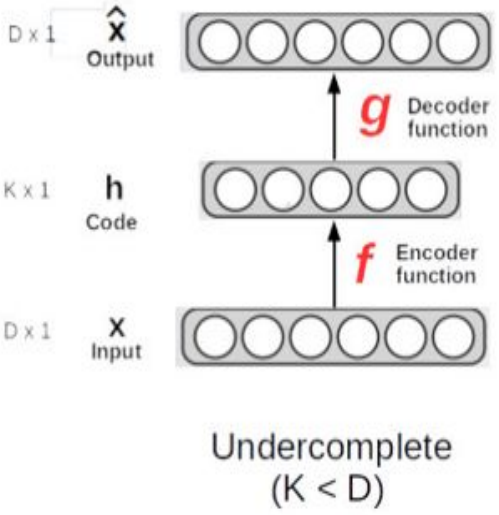
Hyperparameters of Autoencoders:

- There are 4 hyperparameters that we need to set before training an autoencoder:
- Code size: It represents the number of nodes in the middle layer. Smaller size results in more compression.
- Number of layers: The autoencoder can consist of as many layers as we want.
- Number of nodes per layer: The number of nodes per layer decreases with each subsequent layer of the encoder, and increases back in the decoder. The decoder is symmetric to the encoder in terms of the layer structure.
- Loss function: We either use mean squared error or binary cross-entropy. If the input values are in the range [0, 1] then we typically use cross-entropy, otherwise, we use the mean squared error.



Undercomplete Autoencoders

- Copying the input to the output may sound useless, but we are typically not interested in the output of the decoder. Instead, we hope that training the autoencoder to perform the input copying task will result in h taking on useful properties.
- One way to obtain useful features from the autoencoder is to constrain h to have a smaller dimension than x. An autoencoder whose code dimension is less than the input dimension is called undercomplete. Learning an undercomplete representation forces the autoencoder to capture the most salient features of the training data.

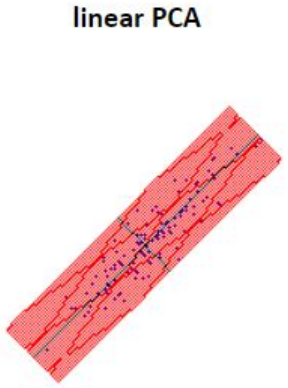


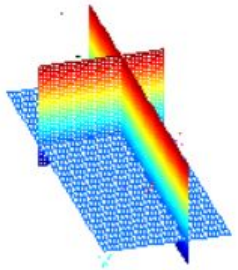
- The learning process is described simply as minimizing a loss function

$$L(x, g(f(x)))$$

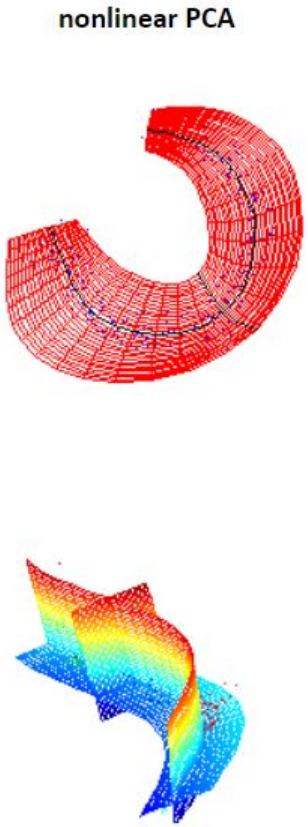
- Where L is a loss function penalizing g(f(x)) for being dissimilar from x, such as the mean squared error.

- When the decoder is linear and L is the mean squared error, an undercomplete autoencoder learns to span the same subspace as PCA. In this case, an autoencoder trained to perform the copying task has learned the principal subspace of the training data as a side effect.



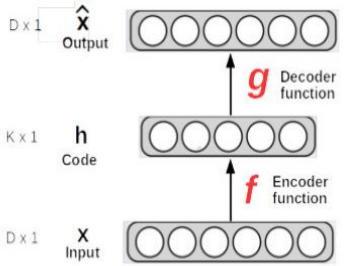


- Autoencoders with nonlinear encoder functions f and nonlinear decoder functions g can thus learn a more powerful nonlinear generalization of PCA.
- Unfortunately, if the encoder and decoder are allowed too much capacity, the autoencoder can learn to perform the copying task without extracting useful information about the distribution of the data.

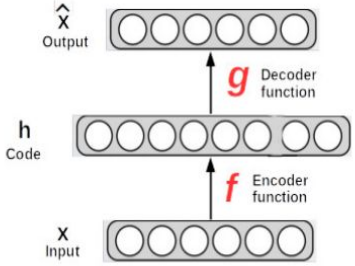


Regularized Autoencoders

- Undercomplete autoencoders, with code dimension less than the input dimension, can learn the most salient features of the data distribution. We have seen that these autoencoders fail to learn anything useful if the encoder and decoder are given too much capacity.
- A similar problem occurs if the hidden code is allowed to have dimension equal to the input, and in the overcomplete case in which the hidden code has dimension greater than the input. In these cases, even a linear encoder and a linear decoder can learn to copy the input to the output without learning anything useful about the data distribution.



Undercomplete
(K < D)



Overcomplete
(K >= D)

- Ideally, one could train any architecture of autoencoder successfully, choosing the code dimension and the capacity of the encoder and decoder based on the complexity of distribution to be modeled. Regularized autoencoders provide the ability to do so.

- Rather than limiting the model capacity by keeping the encoder and decoder shallow and the code size small, regularized autoencoders use a loss function that encourages the model to have other properties besides the ability to copy its input to its output.

- These other properties include:
  - sparsity of the representation,
  - smallness of the derivative of the representation,
  - and robustness to noise or to missing inputs.

- A regularized autoencoder can be nonlinear and overcomplete but still learn something useful about the data distribution, even if the model capacity is great enough to learn a trivial identity function.

Sparse Autoencoders

- A sparse autoencoder is simply an autoencoder whose training criterion involves a sparsity penalty $\Omega(h)$ on the code layer h, in addition to the reconstruction error:

$$L(x, g(f(x))) + \Omega(h)$$

Where g(h) is the decoder output, and typically we have h=f(x), the encoder output.

- Sparse autoencoders are typically used to learn features for another task, such as classification. An autoencoder that has been regularized to be sparse must respond to unique statistical features of the dataset it has been trained on, rather than simply acting as an identity function. In this way, training to perform the copying task with a sparsity penalty can yield a model that has learned useful features as a byproduct.

- Advantages-
- Sparse autoencoders have a sparsity penalty, a value close to zero but not exactly zero.
- Sparsity penalty is applied on the hidden layer in addition to the reconstruction error. This prevents overfitting.
- They take the highest activation values in the hidden layer and zero out the rest of the hidden nodes. This prevents autoencoders to use all of the hidden nodes at a time and forcing only a reduced number of hidden nodes to be used.
- Drawbacks-
- For it to be working, it's essential that the individual nodes of a trained model which activate are data dependent, and that different inputs will result in activations of different nodes through the network.

Denoising Autoencoders

- Rather than adding a penalty Ω to the cost function, we can obtain an autoencoder that learns something useful by changing the reconstruction error term of the cost function.
- Traditionally, autoencoders minimize some function

$$L(x, g(f(x)))$$

Where L is a loss function penalizing g(f(x)) for being dissimilar from x

- A denoising autoencoder (DAE) instead minimizes

$$L(x, g(f(\tilde{x})))$$

Where $\tilde{x}$ is a copy of x that has been corrupted by some form of noise.

- Denoising autoencoders must therefore undo this corruption rather than simply copying their input.

- Denoising autoencoders thus provide yet another example of how useful properties can emerge as a by product of minimizing reconstruction error.

- They are also an example of how overcomplete, high-capacity models may be used as autoencoders as long as care is taken to prevent them from learning the identity function.

Regularizing by Penalizing Derivatives

- Another strategy for regularizing an autoencoder is to use a penalty Ω, as in sparse autoencoders,

$$L(x, g(f(x))) + \Omega(h, x)$$

but with a different form of Ω.

- This forces the model to learn a function that does not change much when x changes slightly. Because this penalty is applied only at training examples, it forces the autoencoder to learn features that capture information about the training distribution.
- An autoencoder regularized in this way is called a contractive autoencoder , or CAE.

Representational Power, Layer Size and Depth

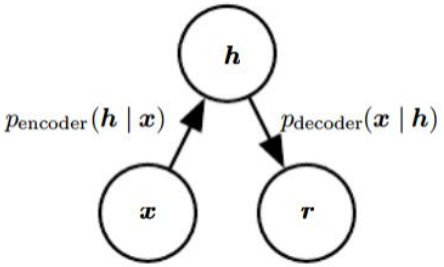
- One major advantage of significant depth is that the universal approximation theorem guarantees that a feedforward neural network with at least one hidden layer can represent an approximation of any function to an arbitrary degree of accuracy, provided that it has enough hidden units
- The Universal Approximation Theorem for neural networks - Bing video
- Although universal approximation theorem states that a single layer is sufficient, there are disadvantages:

1. no of units needed may be too large
2. may not generalize well

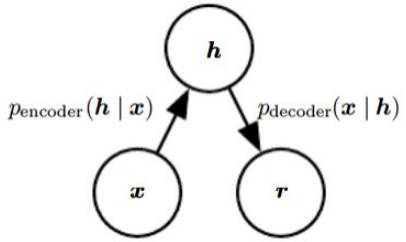
- Depth can exponentially reduce the computational cost of representing some functions.
- Deep autoencoders yield much better compression than corresponding shallow or linear autoencoders

Stochastic Encoders and Decoders

- Autoencoders are just feedforward networks. The same loss functions and output unit types that can be used for traditional feedforward networks are also used for autoencoders.
- we can also generalize the notion of an encoding function f(x) to an encoding distribution p (h | x).



- Both the encoder and decoder are not simple functions but involve a distribution.
- The output is sampled from a distribution $p_{\quad}$. (h | x) for the encoder and $p_{decoder}(x|h)$ for the de



- Any latent variable model $p_{model}(h, x)$ defines a stochastic encoder

  $p_{encoder}(h \mid x) = p_{model}(h \mid x)$

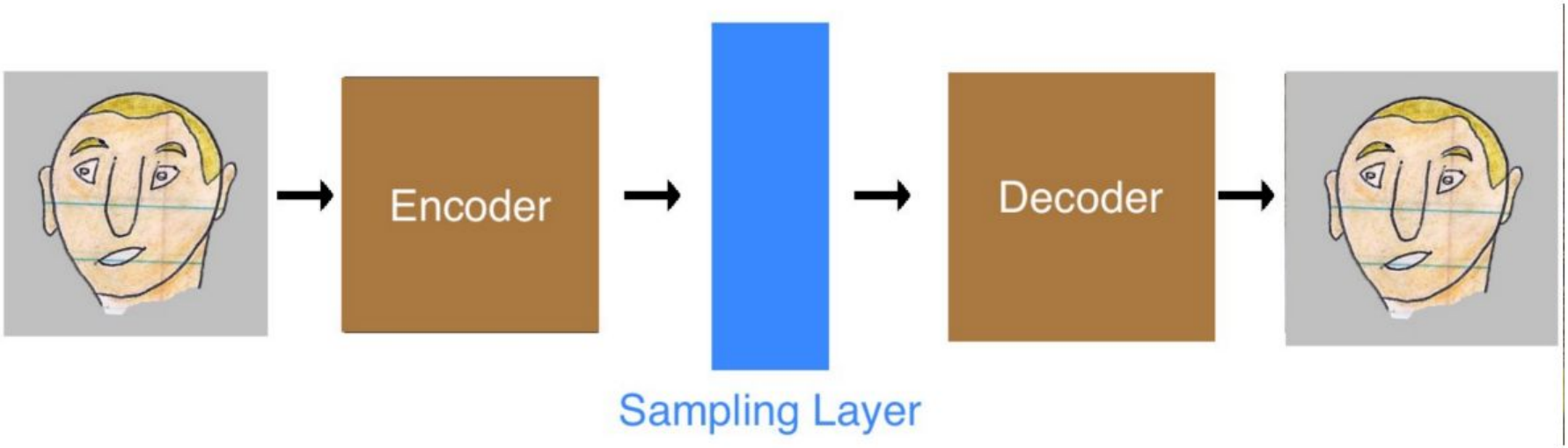
  and a stochastic decoder

  $p_{decoder}(x \mid h) = p_{model}(x \mid h)$.

- In general, the encoder and decoder distributions are not necessarily conditional distributions compatible with a unique joint distribution $p_{model}(x, h)$.
- Alain et al.(2015) showed that training the encoder and decoder as a denoising autoencoder will tend to make them compatible asymptotically

Sampling $p_{\text{model}}(\boldsymbol{h}|\boldsymbol{x})$

$\boldsymbol{x}$

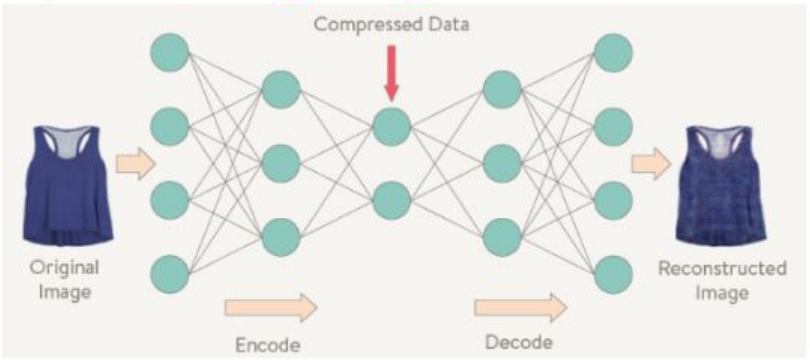


Sampling Layer

$p_{\text{encoder}}(\boldsymbol{h}|\boldsymbol{x})$ $p_{\text{decoder}}(\boldsymbol{x}|\boldsymbol{h})$

Ex: Sampling p(x|h): Deepstyle

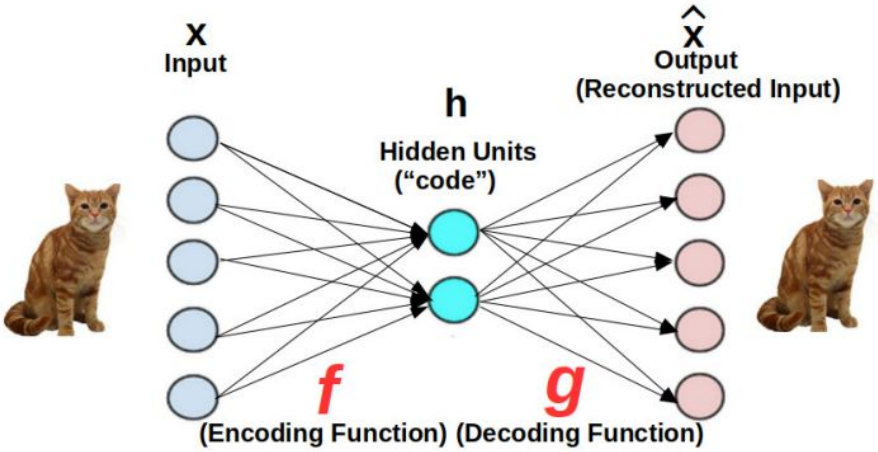
- By iterating neural network through a set of images learn efficient representations.
- Choosing a random numerical description in encoded space will generate new images of styles not seen.





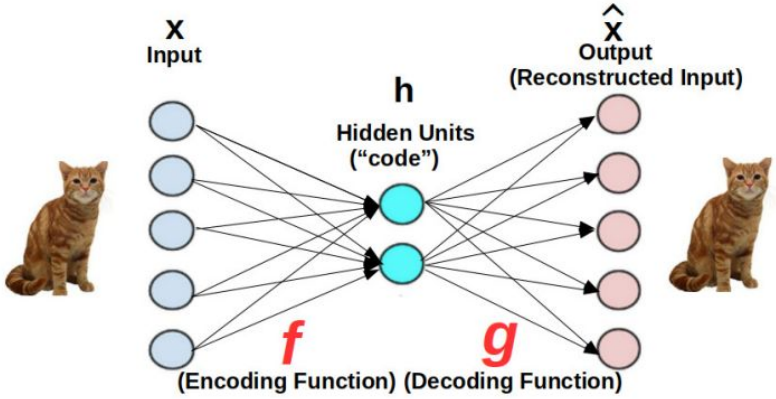
Autoencoder

- Similar to the standard feedforward neural network with a key difference:
- Unsupervised. No “label” at the output layer; Output layer simply tries to “recreate” the input

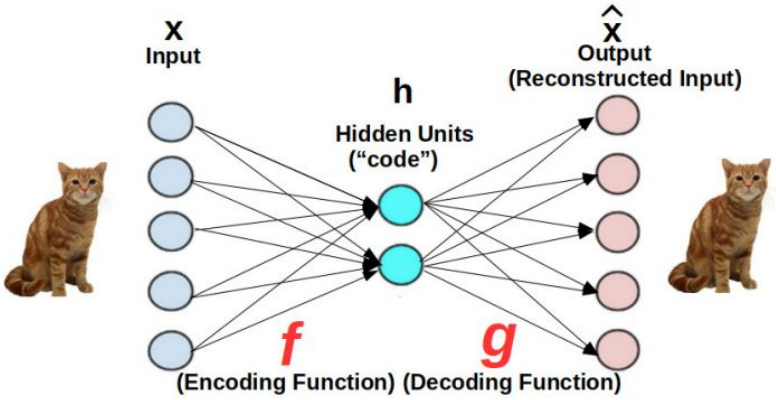


Autoencoder

- Defined by two (possibly nonlinear) mapping functions: Encoding function f , Decoding function g
- h = f (x) denotes an encoding (possibly nonlinear) for the input x
- x^ = g(h) = g(f (x)) denotes the reconstruction (or the “decoding”) for the input x
- For an Autoencoder, f and g are learned with a goal to minimize the difference between x^ and x



- The learned code h = f (x) can be used as a new feature representation of the input x.
- Therefore autoencoders can also be used for “feature learning”.

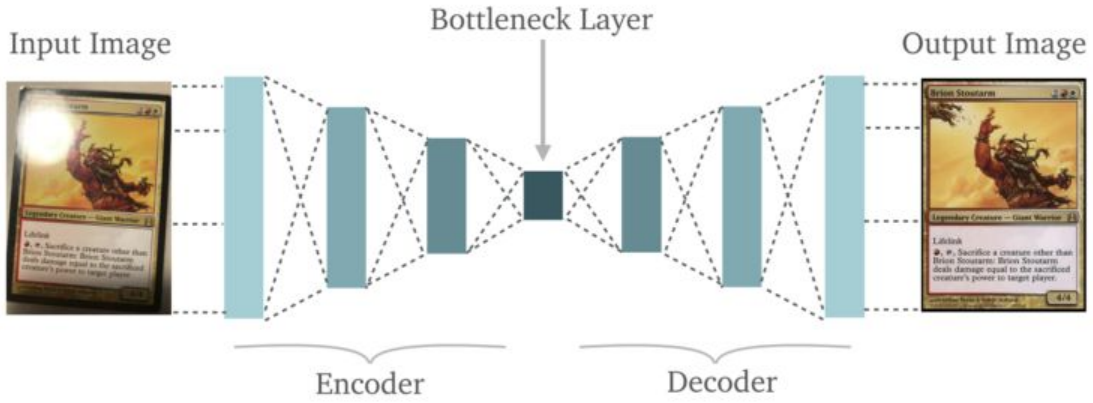


Applications of autoencoders

- Image Coloring
- Autoencoders are used for converting any black and white picture into a colored image. Depending on what is in the picture, it is possible to tell what the color should be



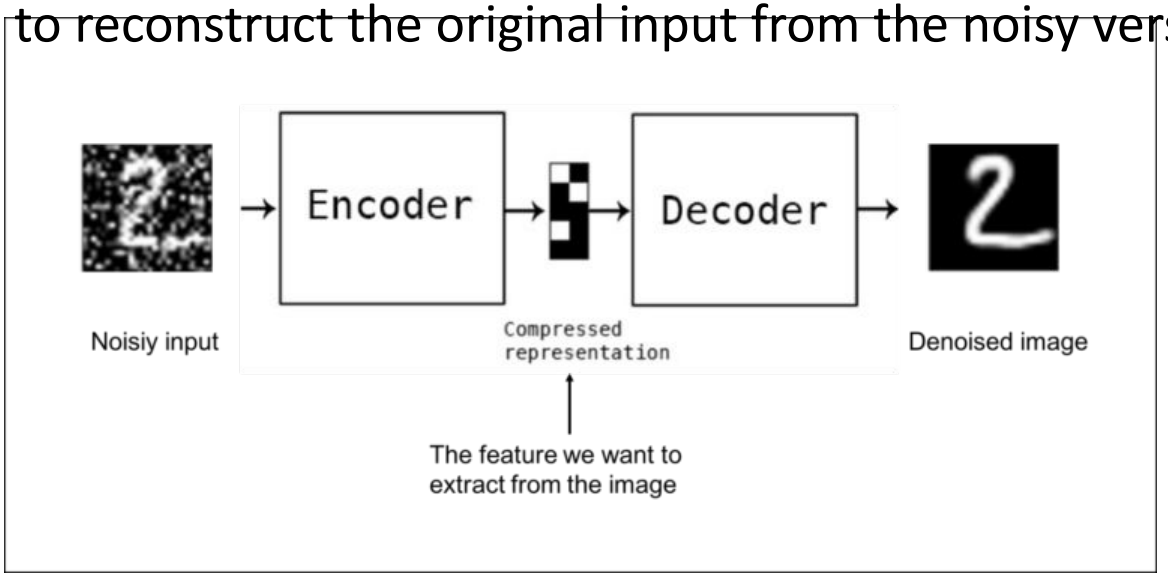
- Feature variation
- It extracts only the required features of an image and generates the output by removing any noise or unnecessary interruption.



- Dimensionality Reduction
- The reconstructed image is the same as our input but with reduced dimensions. It helps in providing the similar image with a reduced pixel value.



- Denoising Image
- The input seen by the autoencoder is not the raw input but a stochastically corrupted version. A denoising autoencoder is thus trained to reconstruct the original input from the noisy version.



- Watermark Removal
- It is also used for removing watermarks from images or to remove any object while filming a video or a movie.



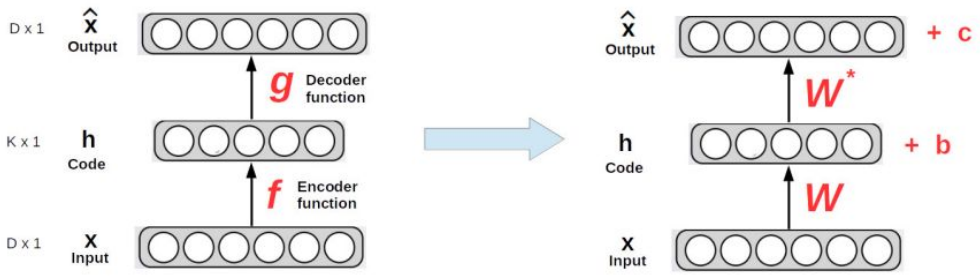
A Simple Autoencoder

- Let's assume a D × 1 input $x \in R^D$, and a single hidden layer with K × 1 code $h \in R^K$
- We can then define a simple linear autoencoder as

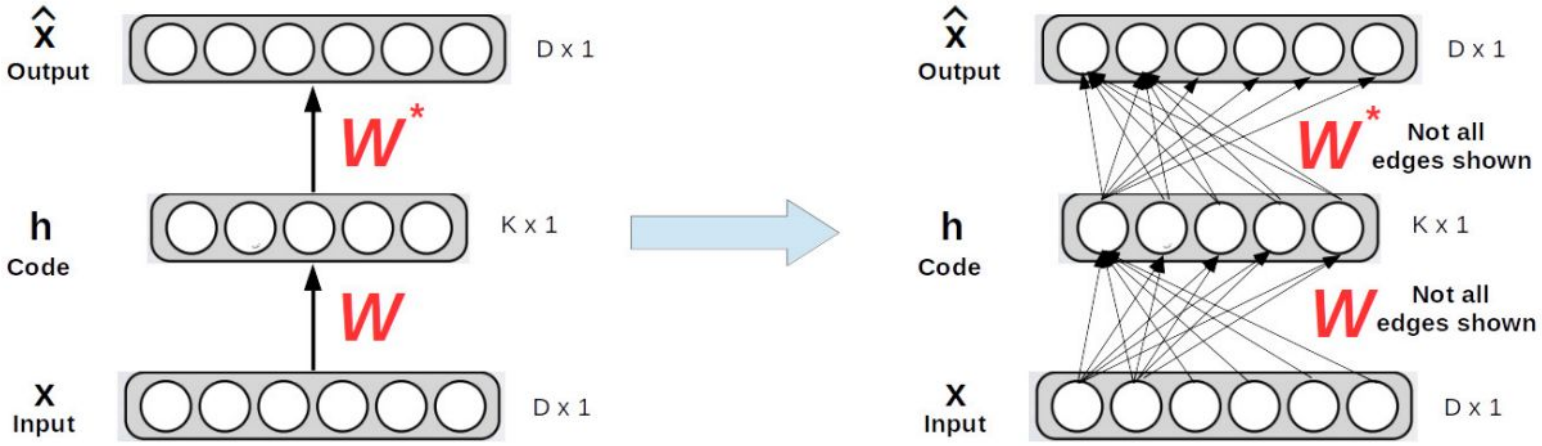
$$h = f(x) = Wx + b$$

$$\hat{x} = g(h) = W^*h + c$$

- where f is defined by $W \in R^{K \times D}$ and $b \in R^{K \times 1}$, g is defined by $W^* \in R^{D \times K}$ and $c \in R^{D \times 1}$
- Note: If we learn f , near
  autoencoder with W onent
  Analysis (PCA)



Autoencoder: Zooming in..



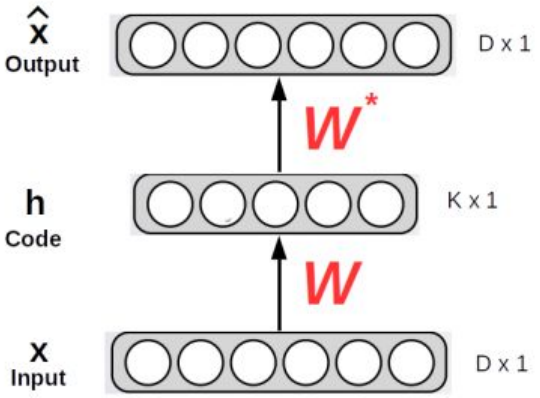
W: K × D matrix of weights of edges between input and hidden layer.
W_{kd} is the weight of edge connecting input layer node d to hidden layer node k.

W^* : D × K matrix of weights of edges between hidden and output layer.
W^*_{dk} is the weight of edge connecting hidden layer node k to output layer node d.

If $W^* = W^T$, the autoencoder architecture is said to have "tied weights"

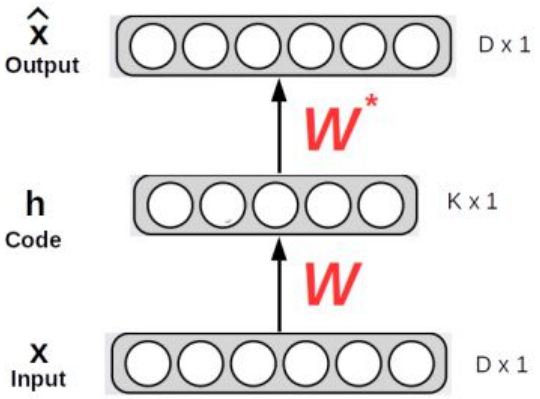
Nonlinear Autoencoders

- The hidden nodes can also be nonlinear transforms of the inputs, e.g.,
- Can define h as a linear transform of x followed by a nonlinearity (e.g., sigmoid, ReLU)
- h = sigmoid(Wx + b)
- where the nonline p(−z)) squashes the real-valued z to lie



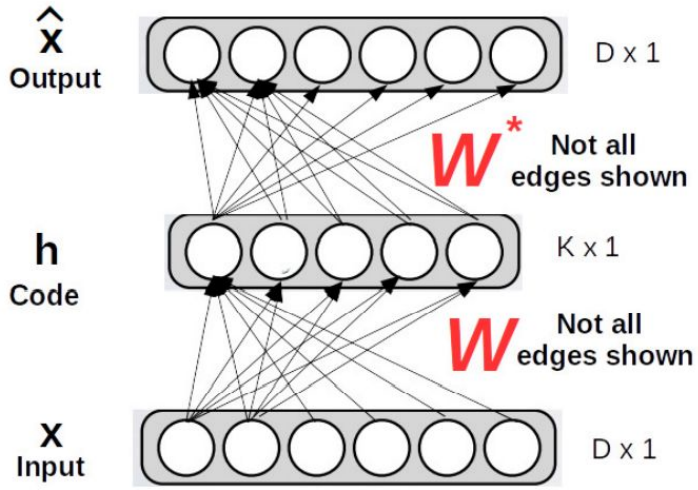
- Note: If inputs $x \in \{0, 1\}^D$ are binary, it may be more appropriate to also define $x^$ as

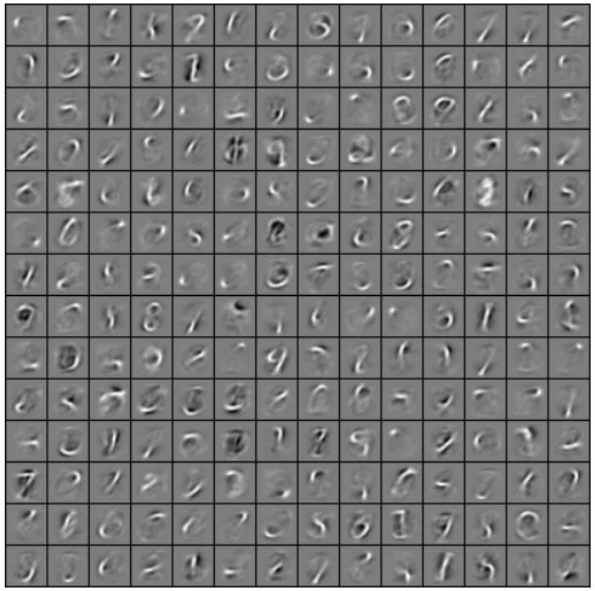
$$x^ = \text{sigmoid}(W^*h + c)$$



What's Learned by an Autoencoder?

- The K × D matrix W learned on digits data. Each tiny block visualizes a row of W.
- Thus W captures the possible "patterns" in the training data.





Training the Autoencoder

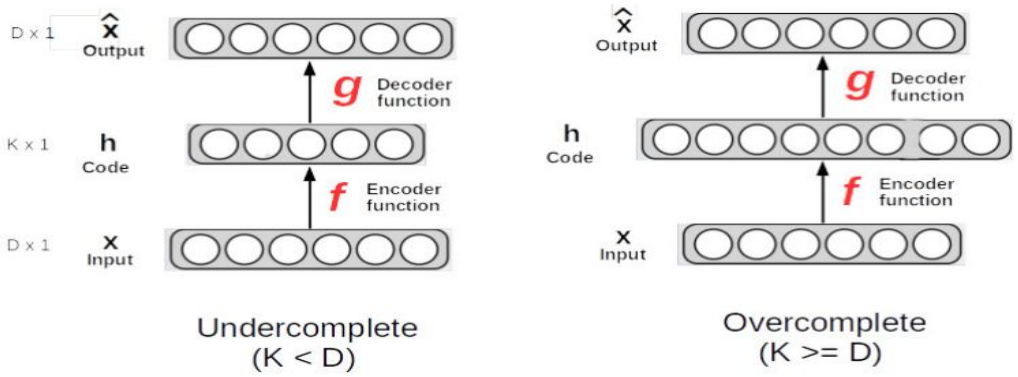
- To train the autoencoder, we need to define a loss function `(x^, x)
- The loss function (a function of parameters W, b, W*, c) can be defined using various ways
- In general, it is defined in terms of the difference between x^ and x (reconstruction error)
- For a single input x = [x1 ... xD] and its reconstruction x^ = [^x1 ... x^D]

$$\ell(\hat{\boldsymbol{x}}, \boldsymbol{x}) = \sum_{d=1}^{D} (\hat{x}_d - x_d)^2 \quad \text{(squared loss; used if input are real-valued)}$$

$$\ell(\hat{\boldsymbol{x}}, \boldsymbol{x}) = -\sum_{d=1}^{D} [x_d \log(\hat{x}_d) + (1 - x_d) \log(1 - \hat{x}_d)] \quad \text{(cross-entropy loss; used if input are binary)}$$

- We find (W, b, W*, c) by minimizing the reconstruction error (summed over all training data)
- This can be done using backpropagation

Undercomplete, Overcomplete, and Need for Regularization



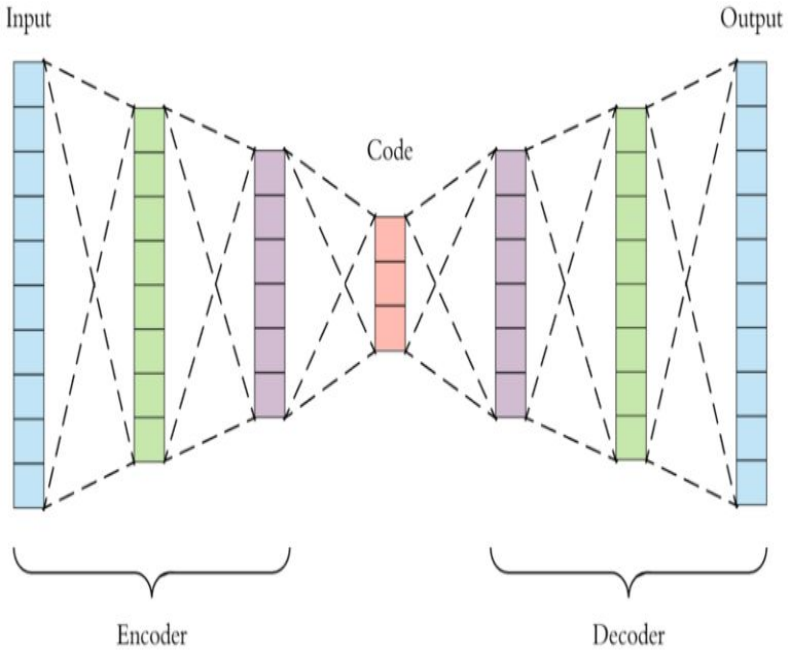
- In both cases, it is important to control the capacity of encoder and decoder
- Undercomplete: Imagine K = 1 and very powerful f and g. Can achieve very small reconstruction error but the learned code will not capture any interesting properties in the data
- Overcomplete: Imagine K ≥ D and trivial (identity) functions f and g. Can achieve even zero reconstruction error but the learned code will not capture any interesting properties in the data
- It is therefore important to regularize the functions as well as the learned code, and not just focus on minimizing the reconstruction error

Properties of Autoencoders:

- Data-specific: Autoencoders are only able to compress data similar to what they have been trained on.
- Lossy: The decompressed outputs will be degraded compared to the original inputs.
- Learned automatically from examples: It is easy to train specialized instances of the algorithm that will perform well on a specific type of input.

Hyperparameters of Autoencoders:

- There are 4 hyperparameters that we need to set before training an autoencoder:
- Code size: It represents the number of nodes in the middle layer. Smaller size results in more compression.
- Number of layers: The autoencoder can consist of as many layers as we want.
- Number of nodes per layer: The number of nodes per layer decreases with each subsequent layer of the encoder, and increases back in the decoder. The decoder is symmetric to the encoder in terms of the layer structure.
- Loss function: We either use mean squared error or binary cross-entropy. If the input values are in the range [0, 1] then we typically use cross-entropy, otherwise, we use the mean squared error.



Undercomplete Autoencoders

- Copying the input to the output may sound useless, but we are typically not interested in the output of the decoder. Instead, we hope that training the autoencoder to perform the input copying task will result in h taking on useful properties.
- One way to obtain useful features from the autoencoder is to constrain h to have a smaller dimension than x. An autoencoder whose code dimension is less than the input dimension is called undercomplete. Learning an undercomplete representation forces the autoencoder to capture the most salient features of the training data.

- The learning process is described simply as minimizing a loss function

$$L(x, g(f(x)))$$

- Where L is a loss function penalizing g(f(x)) for being dissimilar from x, such as the mean squared error.

- When the decoder is linear and L is the mean squared error, an undercomplete autoencoder learns to span the same subspace as PCA. In this case, an autoencoder trained to perform the copying task has learned the principal subspace of the training data as a side effect.

- Autoencoders with nonlinear encoder functions f and nonlinear decoder functions g can thus learn a more powerful nonlinear generalization of PCA.
- Unfortunately, if the encoder and decoder are allowed too much capacity, the autoencoder can learn to perform the copying task without extracting useful information about the distribution of the data.

Regularized Autoencoders

- Undercomplete autoencoders, with code dimension less than the input dimension, can learn the most salient features of the data distribution. We have seen that these autoencoders fail to learn anything useful if the encoder and decoder are given too much capacity.
- A similar problem occurs if the hidden code is allowed to have dimension equal to the input, and in the overcomplete case in which the hidden code has dimension greater than the input. In these cases, even a linear encoder and a linear decoder can learn to copy the input to the output without learning anything useful about the data distribution.

- Ideally, one could train any architecture of autoencoder successfully, choosing the code dimension and the capacity of the encoder and decoder based on the complexity of distribution to be modeled. Regularized autoencoders provide the ability to do so.

- Rather than limiting the model capacity by keeping the encoder and decoder shallow and the code size small, regularized autoencoders use a loss function that encourages the model to have other properties besides the ability to copy its input to its output.

- These other properties include:
  - sparsity of the representation,
  - smallness of the derivative of the representation,
  - and robustness to noise or to missing inputs.

- A regularized autoencoder can be nonlinear and overcomplete but still learn something useful about the data distribution, even if the model capacity is great enough to learn a trivial identity function.

Sparse Autoencoders

- A sparse autoencoder is simply an autoencoder whose training criterion involves a sparsity penalty $\Omega(h)$ on the code layer h, in addition to the reconstruction error:

$$L(x, g(f(x))) + \Omega(h)$$

Where g(h) is the decoder output, and typically we have h=f(x), the encoder output.

- Sparse autoencoders are typically used to learn features for another task, such as classification. An autoencoder that has been regularized to be sparse must respond to unique statistical features of the dataset it has been trained on, rather than simply acting as an identity function. In this way, training to perform the copying task with a sparsity penalty can yield a model that has learned useful features as a byproduct.

Denoising Autoencoders

- Rather than adding a penalty Ω to the cost function, we can obtain an autoencoder that learns something useful by changing the reconstruction error term of the cost function.
- Traditionally, autoencoders minimize some function

$$L(x, g(f(x)))$$

Where L is a loss function penalizing g(f(x)) for being dissimilar from x

- A denoising autoencoder (DAE) instead minimizes

$$L(x, g(f(\tilde{x})))$$

Where $\tilde{x}$ is a copy of x that has been corrupted by some form of noise.

- Denoising autoencoders must therefore undo this corruption rather than simply copying their input.

- Denoising autoencoders thus provide yet another example of how useful properties can emerge as a by product of minimizing reconstruction error.

- They are also an example of how overcomplete, high-capacity models may be used as autoencoders as long as care is taken to prevent them from learning the identity function.

Regularizing by Penalizing Derivatives

- Another strategy for regularizing an autoencoder is to use a penalty Ω, as in sparse autoencoders,

$$L(x, g(f(x))) + \Omega(h, x)$$

but with a different form of Ω.

- This forces the model to learn a function that does not change much when x changes slightly. Because this penalty is applied only at training examples, it forces the autoencoder to learn features that capture information about the training distribution.
- An autoencoder regularized in this way is called a contractive autoencoder , or CAE.

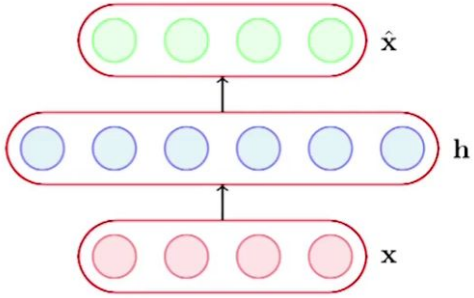
CONTRACTIVE AUTOENCODER

- Contractive autoencoder is an unsupervised deep learning technique that helps a neural network encode unlabeled training data.
- A simple autoencoder is used to compress information of the given data while keeping the reconstruction cost as low as possible.

- A contractive autoencoder also tries to prevent an overcomplete and undercomplete autoencoder from learning the identity function.

- It does so by adding the following regularization term to the loss function:

$$\Omega(\theta) = \|J_{\mathbf{x}}(\mathbf{h})\|_F^2$$



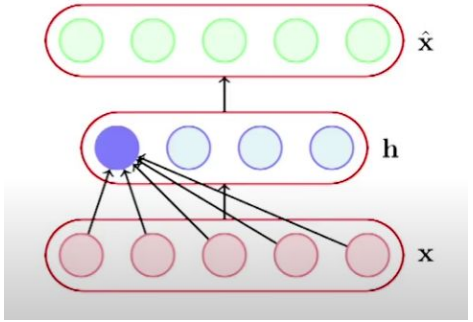
- Frobenius norm of the Jacobian matrix

- Jacobian matrix is some function between two vectors where $x \in R^n$, $h \in R^k$ resulting to n x k Jacobian matrix.

Jacobian Matrix

- If the input has n dimensions and the hidden layer has k dimensions then

$$J_{\mathbf{x}}(\mathbf{h}) = \begin{bmatrix} \frac{\partial h_1}{\partial x_1} & \cdots & \cdots & \cdots & \frac{\partial h_1}{\partial x_n} \\ \frac{\partial h_2}{\partial x_1} & \cdots & \cdots & \cdots & \frac{\partial h_2}{\partial x_n} \\ \vdots & & \ddots & & \vdots \\ \frac{\partial h_k}{\partial x_1} & \cdots & \cdots & \cdots & \frac{\partial h_k}{\partial x_n} \end{bmatrix}$$



- Jacobian is basically the derivative of vector h with respect to the vector x.
- The (j,l) entry of the Jacobian captures the variations in the output of the lth neuron with small variation in the jth neuron.

What does the Frobenius norm capture?

- Sum of the square of all the elements in the matrix.
- Basically it is how much these elements vary with respect to the input and we are just taking the square of it.

$$\|J_{\mathbf{x}}(\mathbf{h})\|_F^2 = \sum_{j=1}^{n}\sum_{l=1}^{k}\left(\frac{\partial h_l}{\partial x_j}\right)^2$$

What is the intuition behind this?

- We add this regularization term with the loss function to minimize the loss function.
- min L(θ) + Ω (θ)

$$\Omega(\theta) = \|J_{\mathbf{x}}(\mathbf{h})\|_F^2$$

$$\|J_{\mathbf{x}}(\mathbf{h})\|_F^2 = \sum_{j=1}^{\cdot\cdot} \sum_{l=1}^{\cdot\cdot} \left(\frac{\partial h_l}{\partial x_j} \right)^2$$

- Ideally we want Frobenius norm to be 0. But this will never take place because there is a tradeoff between L(θ) and Ω (θ). If we make Ω (θ) = 0 then L(θ) will be very high.

- Consider $\frac{\partial h_1}{\partial x_1}$, what does it mean if $\frac{\partial h_1}{\partial x_1} = 0$
- Normally whenever a change is made in x_1 we expect that change to be captured by h_1. But in this case even if there is a slight change in x_1 its not captured by h_1 as the derivative = 0.
- But doesn't this contradict our goal of minimizing L(θ) which requires h to capture variations in the input.
- By putting these two contradictive objectives against each other we ensure that h is sensitive to only very important variations as observed in the learning data.
- L(θ) : Capture important variations in the data.
- Ω(θ): Don't capture variations in data.
- Tradeoff : Capture only very important variations in the data.

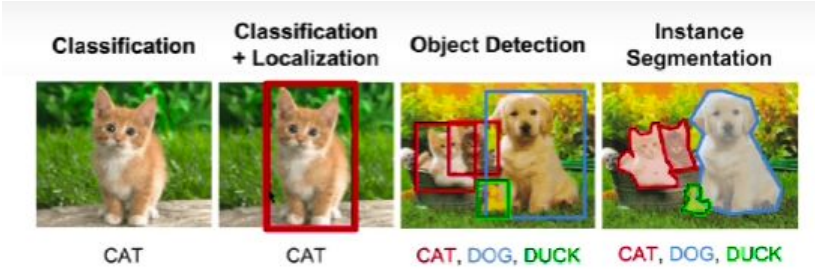
Example

- Consider the variations in the data along directions u1 and u2.
- It makes sense to maximize a neuron to be sensitive to variations along u1.
- At the same time it makes sense to inhibit a neuron from being sensitive to variations along u2 (as there seems to be small noise and unimportant for reconstruction).
- By doing so we can balance between the contradicting goals of good reconstruction and low sensitivity.

Deep Generative Models

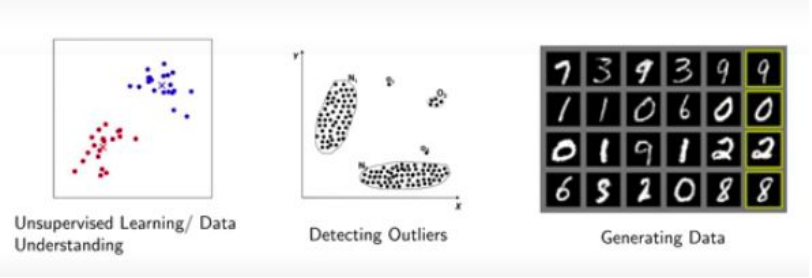
- Deep Generative Models includes topics such as Generative Adversarial Networks (GANs) and Variational Auto Encoders (VAEs).

- Lets understand where Deep Generative Models fit into the scheme of ML methods.
- Supervised Learning
- Examples :
  - Classification
  - Classification with localization
  - Object detection
  - Instance Segmentation

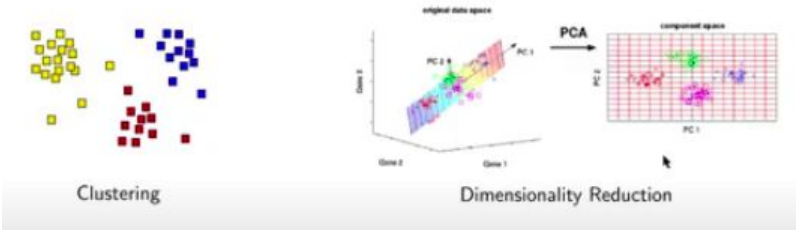


- Output of each is different from each other which resulted in different architectures, loss functions etc.

- Unsupervised learning :
  - Understanding about data
  - Detecting outliers
  - Generating data from the past data.

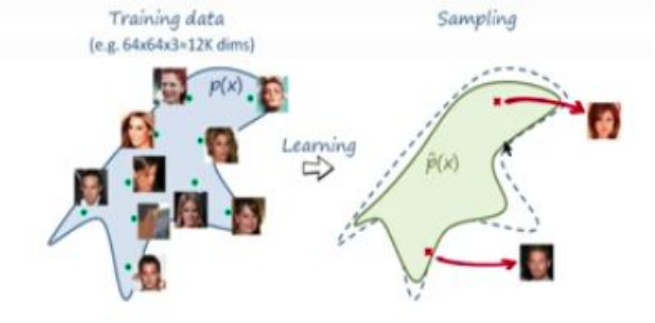


- Examples :
  - Clustering : K means , DBSCAN
  - Dimensionality Reduction : PCA



- The most difficult part is to get annotations/labels for data, which depends on domain can be challenging.
- It requires experts to participate and may be a very costly affair both in terms of money and time.

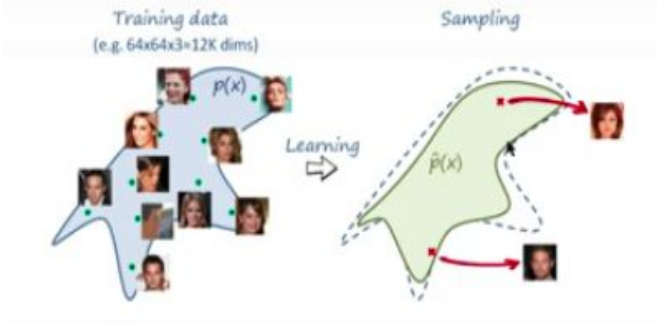
Generative Models



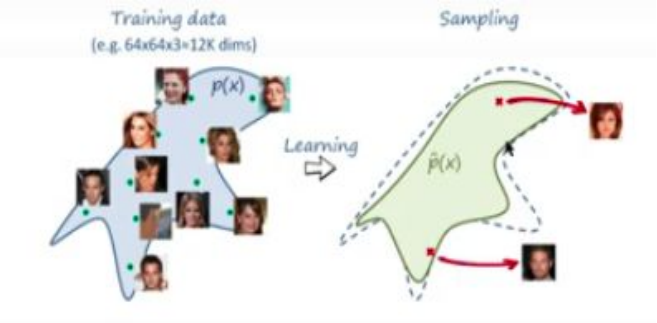
- Its another variant of unsupervised learning.
- Goal : Generate data samples similar to the ones in the training set.
- The datapoints lie on some manifold.
- Manifold corresponds to a locally Euclidean surface, which can be looked at as intrinsic dimensionality of data.
- So, if data lies on a certain manifold, which corresponds to a distribution, We learn using some method the model, which can then sample from a distribution that the model has learned and generate new datapoints.

What does it mean, mathematically?

- Assume training data X = {x1, x2, x3,….,xn} comes from an underlying distribution $P_D(x)$ and a generative model samples data from a distribution $P_M(x)$.
- Our aim is to minimize some notion of distance between $P_D(x)$ and $P_M(x)$.



Generative Models : How to learn?



- Aim: To minimize some notion of distance bet How?
- Consider an approximating distribution $P_M(x)$ coming from a family of distribution M, i.e., we need to find the best distribution in M, parametrized by θ, which minimizes the distance between $P_D(x)$ and $P_M(x)$.
- So the objective function will be :

$$\theta^* = \arg min_{\theta \in M} \; dist\,(P_\theta\,, P_D)$$

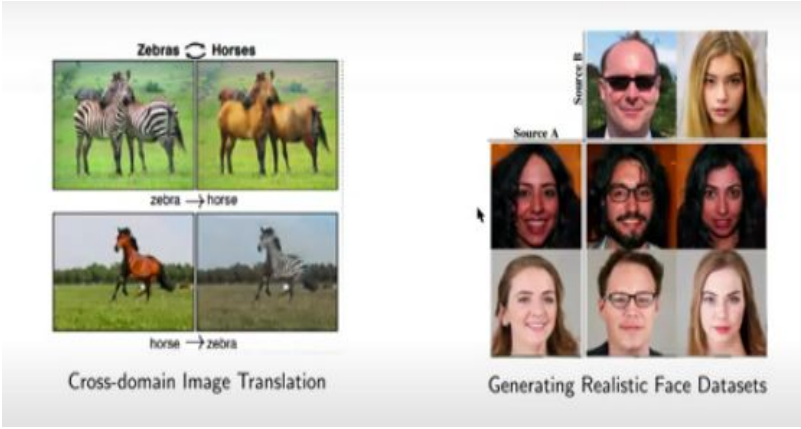
- We can use KL-divergence as the distance function as its used to measure the distance between two probability distributions.

Why should we learn Deep Generative Models?

Applications:

- Image Super Resolution
- Image colorization
- Cross domain image translation
- Generating realistic Face detection
- Augmenting datasets
- Mixed reality application – Virtual Try-on

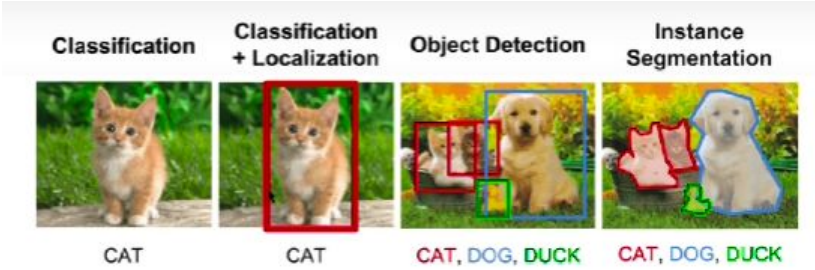




Deep Generative Models

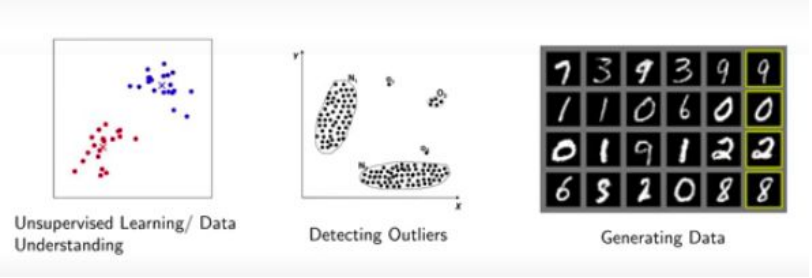
- Deep Generative Models includes topics such as Generative Adversarial Networks (GANs) and Variational Auto Encoders (VAEs).

- Lets understand where Deep Generative Models fit into the scheme of ML methods.
- Supervised Learning
- Examples :
  - Classification
  - Classification with localization
  - Object detection
  - Instance Segmentation

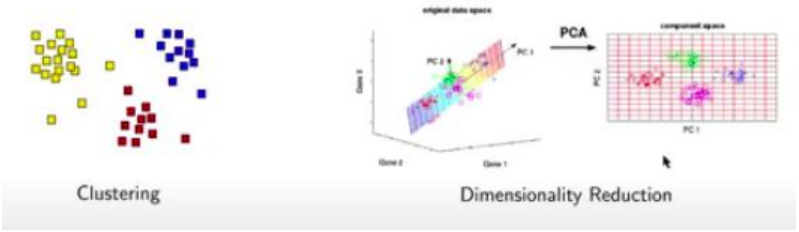


- Output of each is different from each other which resulted in different architectures, loss functions etc.

- Unsupervised learning :
  - Understanding about data
  - Detecting outliers
  - Generating data from the past data.

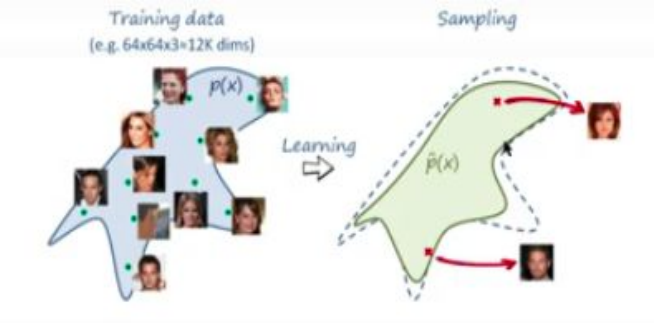


- Examples :
  - Clustering : K means , DBSCAN
  - Dimensionality Reduction : PCA



- The most difficult part is to get annotations/labels for data, which depends on domain can be challenging.
- It requires experts to participate and may be a very costly affair both in terms of money and time.

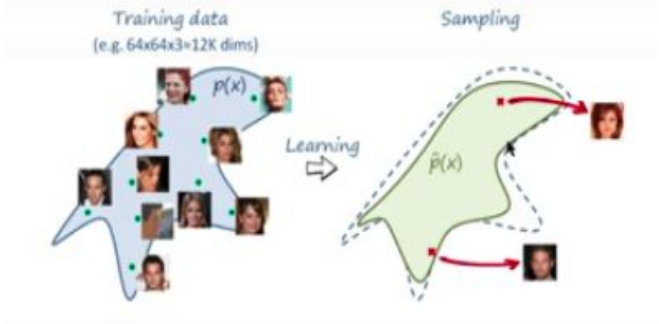
Generative Models



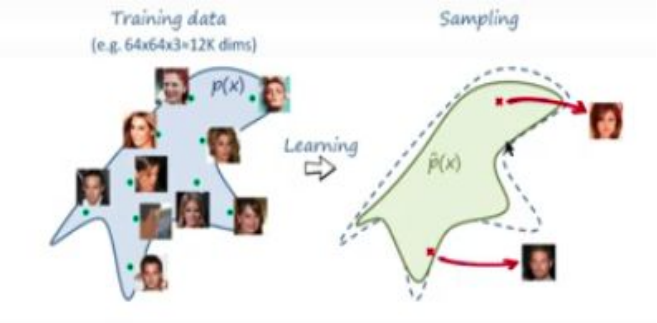
- Its another variant of unsupervised learning.
- Goal : Generate data samples similar to the ones in the training set.
- The datapoints lie on some manifold.
- Manifold corresponds to a locally Euclidean surface, which can be looked at as intrinsic dimensionality of data.
- So, if data lies on a certain manifold, which corresponds to a distribution, We learn using some method the model, which can then sample from a distribution that the model has learned and generate new datapoints.

What does it mean, mathematically?

- Assume training data X = {x1, x2, x3,….,xn} comes from an underlying distribution $P_D(x)$ and a generative model samples data from a distribution $P_M(x)$.
- Our aim is to minimize some notion of distance between $P_D(x)$ and $P_M(x)$.



Generative Models : How to learn?



- Aim: To minimize some notion of distance bet How?
- Consider an approximating distribution $P_M(x)$ coming from a family of distribution M, i.e., we need to find the best distribution in M, parametrized by θ, which minimizes the distance between $P_D(x)$ and $P_M(x)$.
- So the objective function will be :

$$\theta^* = \arg min_{\theta \in M} \; dist\,(P_\theta\,, P_D)$$

- We can use KL-divergence as the distance function as its used to measure the distance between two probability distributions.

Why should we learn Deep Generative Models?

Applications:

- Image Super Resolution
- Image colorization
- Cross domain image translation
- Generating realistic Face detection
- Augmenting datasets
- Mixed reality application – Virtual Try-on



